IERG4320/IEMS5726: Data Science in Practice

Lecture 6: Data Modeling I

Prof. YAN Wenjing, Dr. Danny Yip
Department of Information Engineering

香港中文大學
The Chinese University of Hong Kong

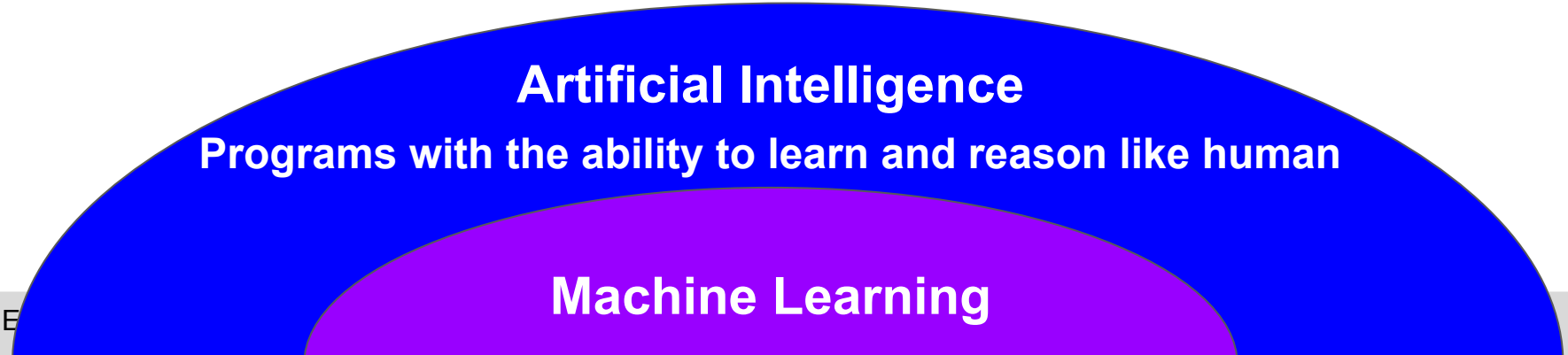
Outline

1. Introduction to Machine Learning (ML)
2. Basic Concepts
3. Classification Algorithms
4. Evaluation and Diagnosing
5. Model Comparisons and Parameter Tuning
6. Regression Algorithms and its Evaluation

All rights are reserved to Prof. YAN Wenjing, Dr. Danny YIP,
The Department of Information Engineering, CUHK

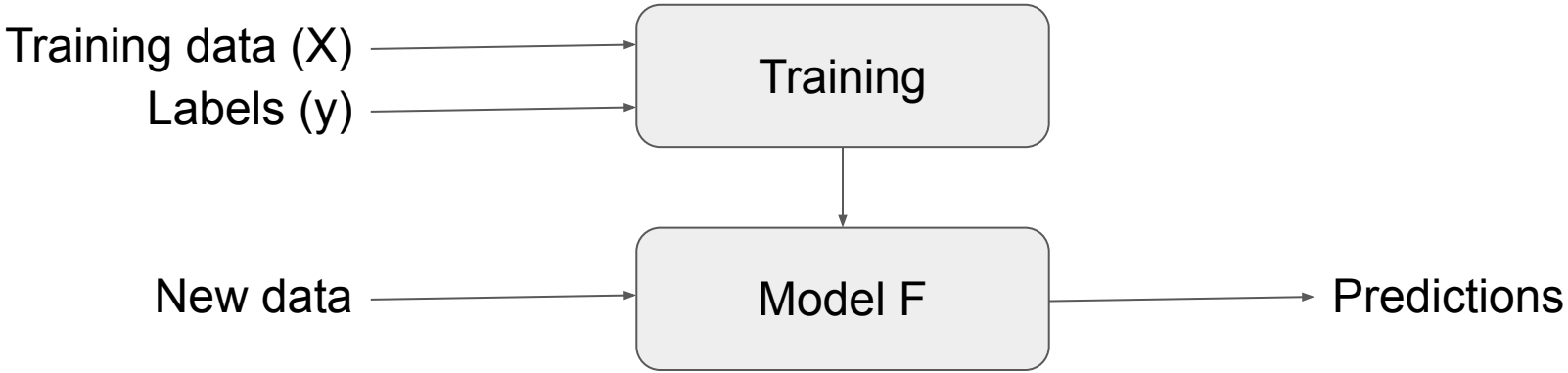
Machine Learning (ML)

- A computer program generates an input given and output, given some **pre-defined function(s)**
- ML aims at **learning** a function that maps the inputs to the outputs
- Instead of having a programmer writing down the **logic**, we let the computer **learn** from the data
- Given historical data, we train a **model** to generate predictions on **future or unseen** inputs



Machine Learning, using Mathematical language

- Given some input X and ouptut y, find a function F(X) that maps X to y.
- Example: given (location, size) (X), predict the price of a house (y).
- Another example: give (previously watched movies) (X), predict the next movie(s) that will be watched (y).



Case Study: Spam Email Detection

- Let's consider the task of detecting spam or junk emails.
- Examples of spam emails:
  - scams
  - phishing
  - mass email marketing / advertisement
  - and many more
- How would you approach this problem?
- (It is a **classification** task: for each email, we want to label it as "spam" or "non-spam")

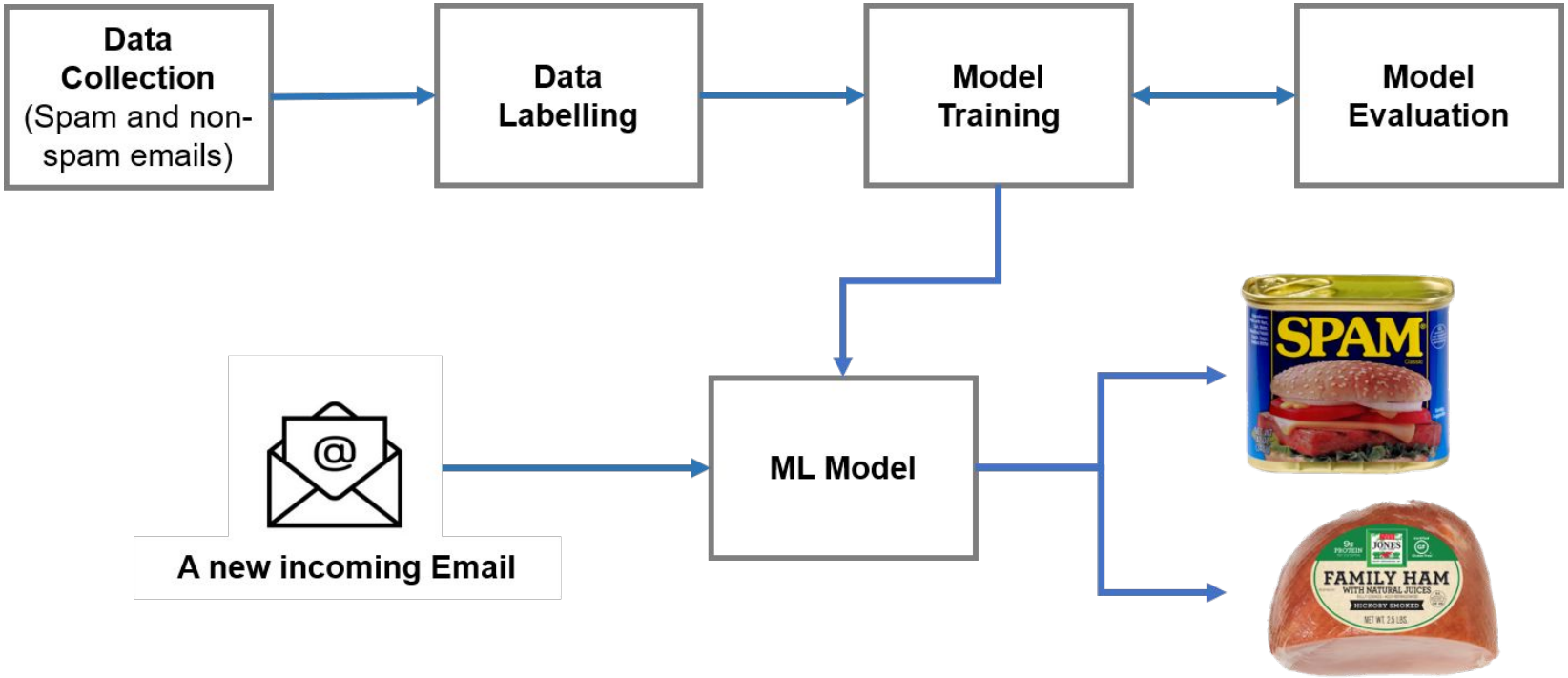
Non-ML approach for Spam Email Detection

- To classify incoming emails, we need to find some patterns that are unique in each classes
- Example of useful patterns:
  - Name of the sender of the email
  - Key words/phrases in the subject (e.g. "You have won a lottery!")
  - A lot of words in capital letters in the content
  - … etc
- You then write a program to look for these patterns in a email received
- **Done?**

Limitation of a non-ML Approach

- You may need to have **a lot of** rules
- Rules too general may give you **false positives**
- Rules too specific will be useless if the spammers make slight changes
- How do you **weight** the importance of each rule?
- … etc

ML Approach for Spam Email Detection



Advantages of using ML

- Learn the important patterns **automatically** from the training data
- An ML algorithm can go through **a large number of examples** to learn patterns
- Able to discover the **relative importance** of different patterns
- Model can be **re-trained** when the data has changed

- Can you think of any weakness of ML?

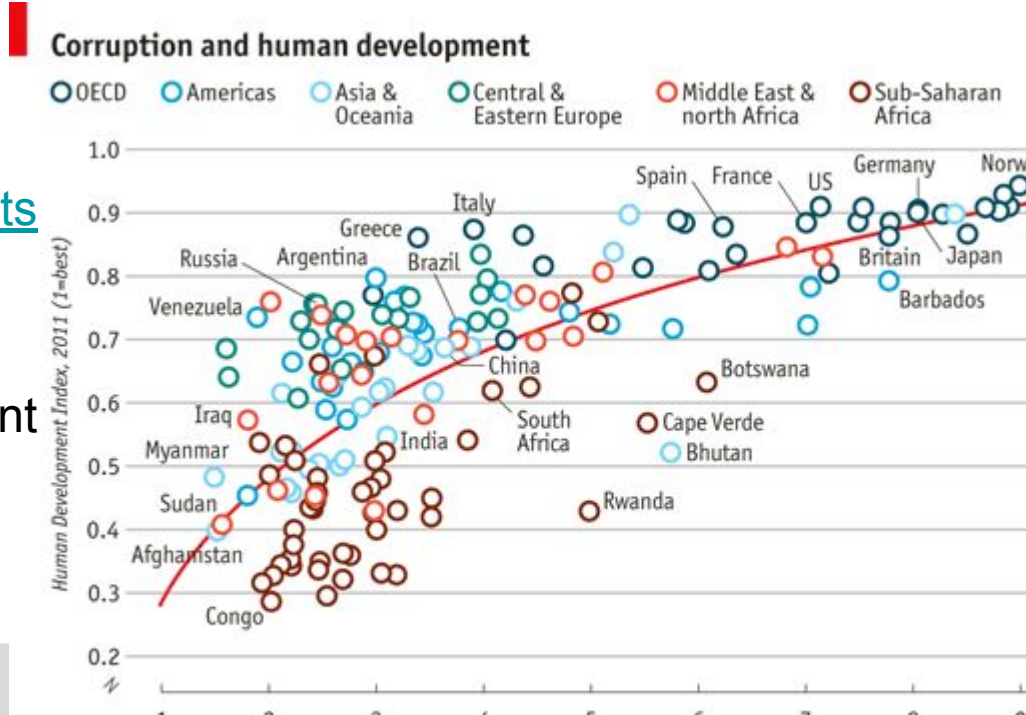
Basic Machine Learning Problems

- **Supervised Learning**: You have labelled data for computer to learn from
  - Regression / Classification
- **Unsupervised Learning**: You don't have labelled data, but you want to find patterns in the data
  - Clustering / Dimensionality Reduction
- **Semi-supervised Learning**: You have a small amount of labeled data, but a large amount of unlabeled data during training
  - Generative Models, Low-density Separation
- **Reinforcement Learning**: You have an environment stated in the form of Markov decision process (MDP). You can search in the environment
  - Search algorithm, e.g. brute force, direct policy search
  - RL algorithm: Q-learning, Deep Q Network

Regression

- In **regression**, we want to predict the value of a **continuous** variable based on some inputs

- Examples:
  - Predict the price of a house
  - Predict the number of products sold in a shop
  - Predict the number of users who will click an advertisement
  - Predict the price of a product given its descriptions

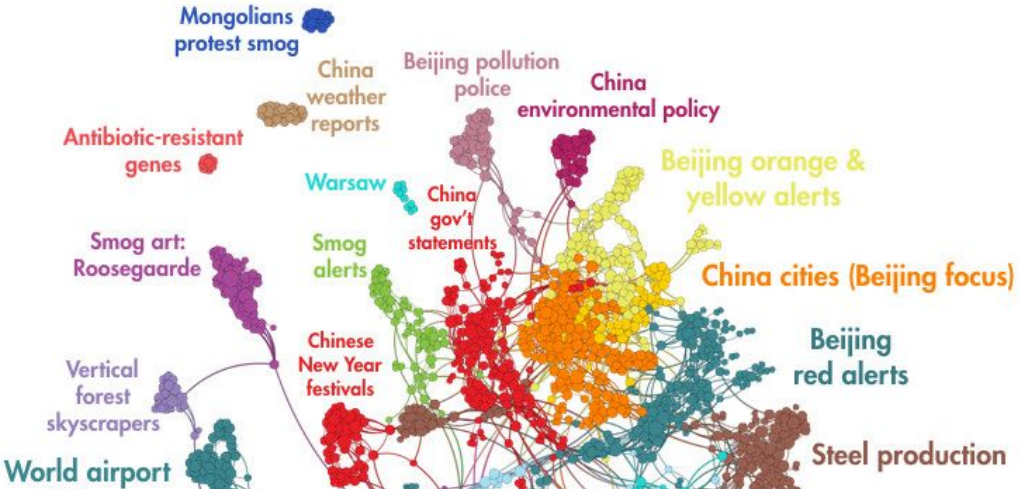


Classification

- Categorizing an input sample into one of the **pre-defined** classes
- Examples:
  - Predict whether a person will survive a shipwreck
  - Detect the sentiment of a movie review
  - Predict whether a driver will initiate an insurance claim
- In fact, a lot of problems can be defined as **classification problems**

Unsupervised Learning / Clustering

- To identify hidden patterns in a dataset without labels
- Examples:
- Understand what products are usually bought together (Basket Analysis)
- Understand major topics of news articles (e.g. News Clustering)
- Identifying communities in social networks
- Clustering can be used to perform dimensionality reduction



Semi-supervised Learning

- In reality, we only have labels for a small set of data.
- Examples:
- To classify the types of the text documents.

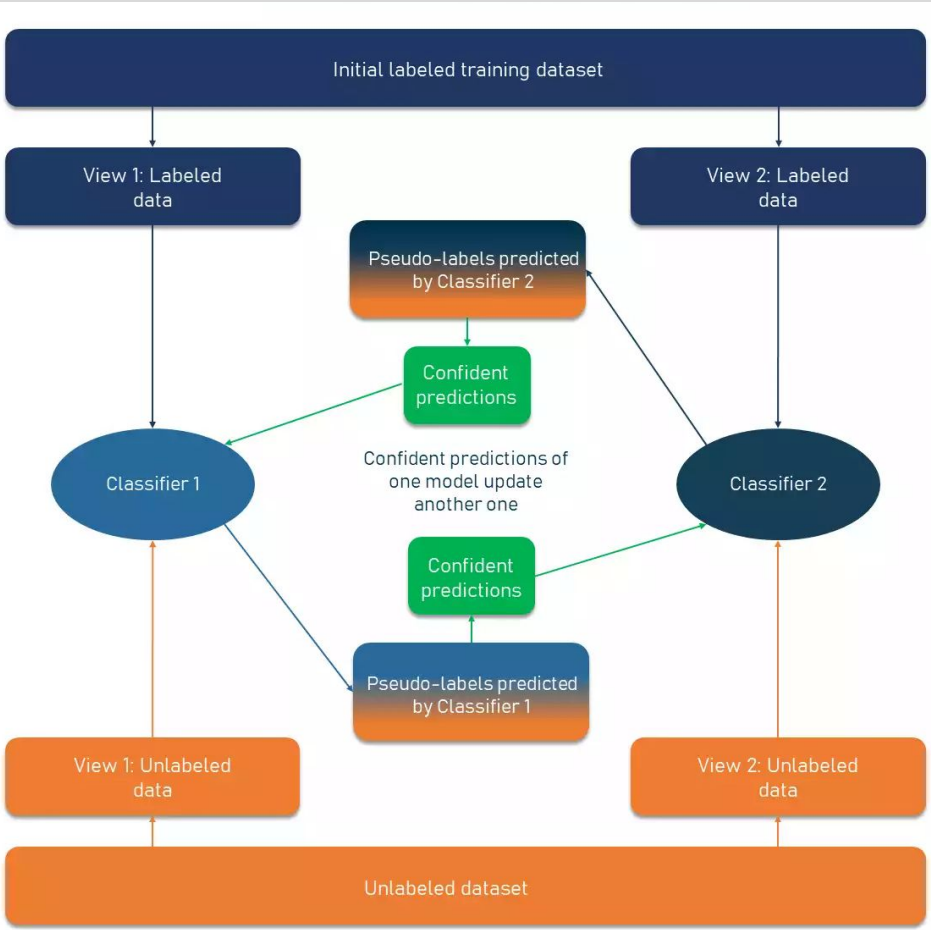
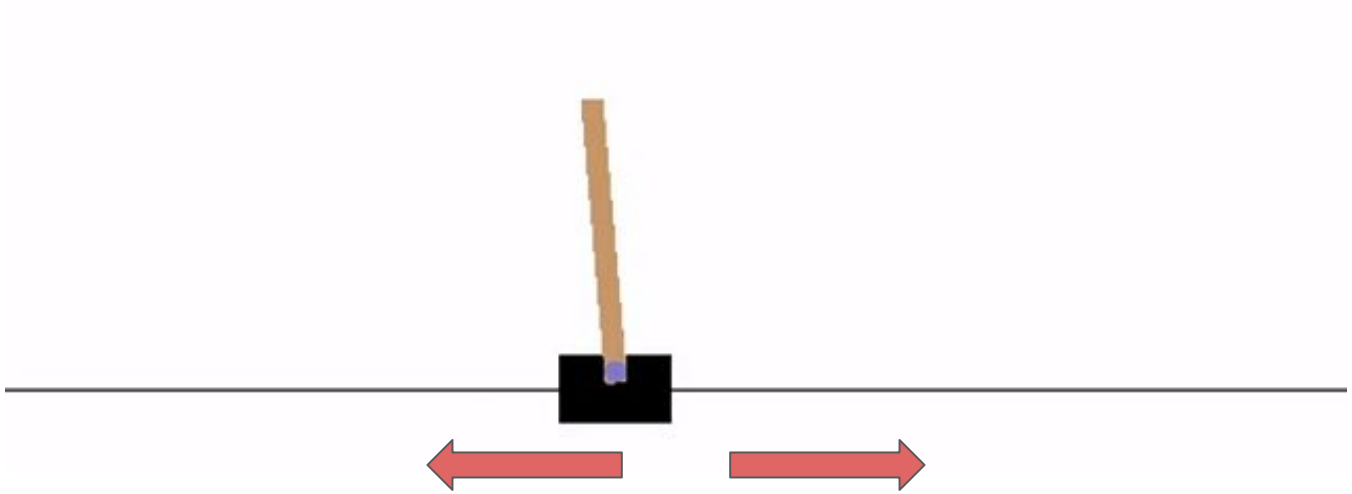


Image credit: https://www.altexsoft.com/blog/semi-supervised-learning/

Reinforcement Learning

- Deep Q Learning (DQN) Agent on the CartPole-v0 Task
- The agent has to decide between two actions - moving the cart left or right
- Goal: The pole attached to it stays upright
- It will be covered in the last week of the course.



Reference: https://pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

Classifications

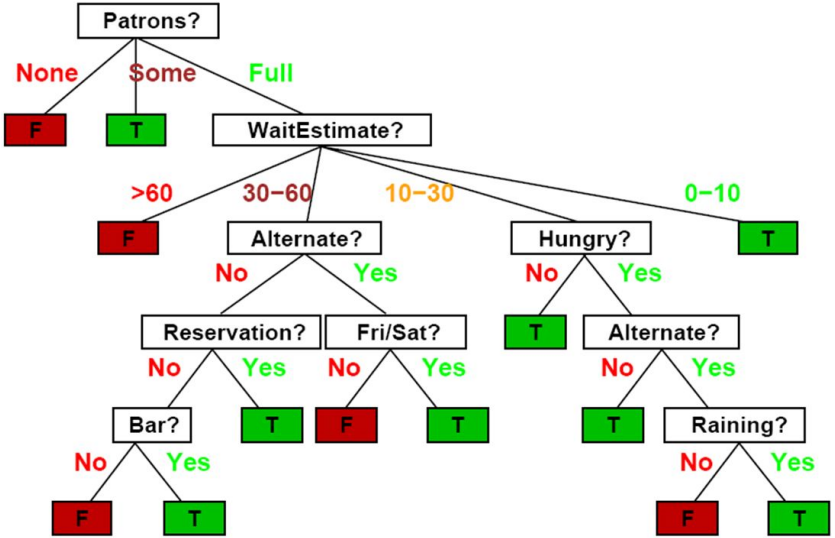
Some Top Classification Algorithms

1. Decision Tree
2. Random Forest
3. Support Vector Machine (SVM)
4. K-Nearest Neighbour (KNN)
5. Artificial Neural Network (ANN)

Decision Tree

$$I\left(\frac{p}{p+n},\frac{n}{p+n}\right)=-\frac{p}{p+n}\log_2\frac{p}{p+n}-\frac{n}{p+n}\log_2\frac{n}{p+n}$$

- A decision tree takes as input an object or situation described by **a set of properties (attributes)**, and outputs a **yes/no "decision"**. Decision trees therefore represent Boolean functions.
- Functions with a larger range of outputs can also be represented, but for simplicity usually stick to the Boolean case.
- **Internal node = a test**; **branches** are labeled with **possible values** of the test. Each leaf node specifies the Boolean value result if reached.

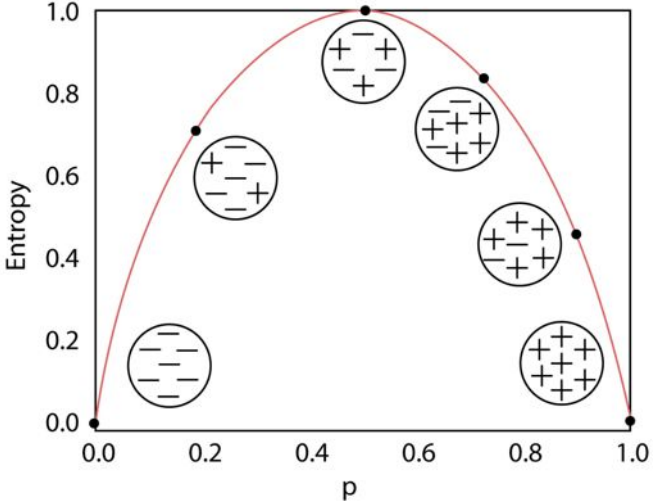


A decision tree for deciding whether to wait for a table

Visualization of Entropy, Gini Index

- The lower the Gini Index, the better the lower the likelihood of misclassification

$$I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}, \quad Gini = 1 - \sum_{i=1}^{j} P(i)^2$$



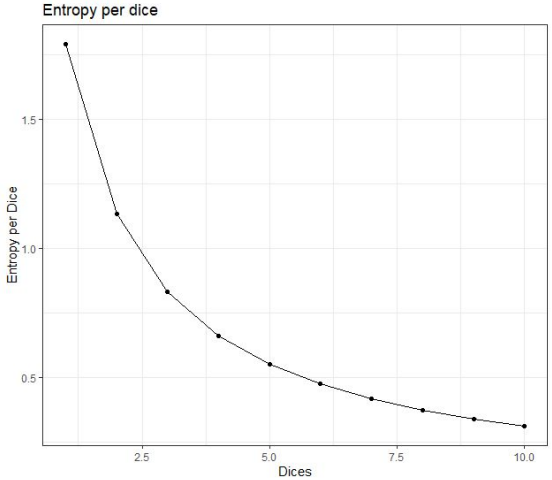


Image credit: Quantinsti, Data Science Central

Decision Tree: Load Libraries

```python
import pandas as pd
# Import Decision Tree Classifier
from sklearn.tree import DecisionTreeClassifier
# Import train_test_split function
from sklearn.model_selection import train_test_split
# Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics
# visualize decision tree, installation:
# conda install -c anaconda graphviz --yes
# conda install -c conda-forge pydotplus --yes
from sklearn.tree import export_graphviz
from six import StringIO
from IPython.display import Image
import pydotplus

```

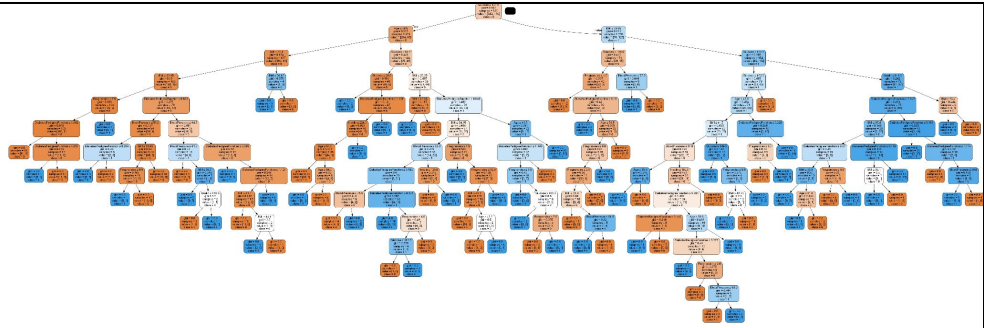
Decision Tree: Prepare Data

```
col_names = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']
# load dataset
pima = pd.read_csv("pima-indians-diabetes.csv")
pima = pima.iloc[1:]
# feature selection, split dataset in features and target variable
feature_cols = ['Pregnancies', 'Insulin', 'BMI',
'Age','Glucose','BloodPressure','DiabetesPedigreeFunction']
X = pima[feature_cols] # Features
y = pima.Outcome       # Target variable
# Split dataset into training set and test set, 70% training and 30% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=1)
```

Reference: https://www.datacamp.com/tutorial/decision-tree-classification-python

Decision Tree: Build and Evaluate Model, Visualize Tree

```
# build decision tree model
# Create Decision Tree classifier object
model = DecisionTreeClassifier()
# Train Decision Tree Classifier
model = model.fit(X_train,y_train)
# Predict the response for test dataset
y_pred = model.predict(X_test)
# evaluate the model
# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
# Accuracy: 0.7056277056277056
dot_data = StringIO()
export_graphviz(model,out_file=dot_data,filled=True,rounded=True,class_names=['0','1'],
                special_characters=True,feature_names = feature_cols)
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
graph.write_png('diabetes.png')
Image(graph.create_png())
```



```
Accuracy: 0.7402597402597403
```

Evaluation

Evaluation

- **Evaluation** is an important step in any machine learning project
- Purposes:

1. Understand the **quality of predictions** of a trained model
2. Understand when does a model perform well or badly
3. Understand the **limitation** of the model

- How:

1. Decide on a suitable **metric(s)** for your problem
2. Pick an algorithm and train a model
3. Compute the metric on the predictions on a test dataset

Metrics

- When evaluating the performance of a model, we need to have:
  - **ground truths**: the correct answer / the true labels of the inputs
  - **metric**: a measure of how good the predictions are compared to the ground truths
- In the following, we focus on:
  - Metrics for **regression** tasks: usually a function that reflects how the **difference** between the predicted values and the true values
  - Metrics for **classification** tasks: usually a function that reflects how often the predicted values are the same as the true values
- Ref: Model evaluation: quantifying the quality of predictions

Metrics for Classification

- In classification, things are different:
  - The labels or classes do not lie on a continuous spectrum
  - Things are discrete: predictions are either **correct or incorrect**
  - We cannot measure **distance** between two labels
- In the following, we will discuss some important metrics for classification:
  - Accuracy
  - True/False positives/negatives
  - Precision and Recall
  - Area Under the ROC Curve

Metrics for Classification - Accuracy

- A straight-forward metric for classification is **accuracy**
- Defined by **number of correct predictions** / **total number of predictions**
- For example:

  y = (1,0,0,0,1,1,1,0,0)

  ŷ = (1,1,1,0,0,1,1,0,0)
- Then, accuracy is **6/9 = 66.7%**
- This can also be applied to multi-class classification
- What is the limitation of the **accuracy score**?

```
from sklearn import metrics
y_test = [1,0,0,0,1,1,1,0,0]
y_pred = [1,1,1,0,0,1,1,0,0]
metrics.accuracy_score(y_test, y_pred)
```

```
0.6666666666666666
```

Metrics for Classification - T/F Positives/Negatives

- To have a better understanding of how well a model performs in predicting different classes
- Consider a binary classification task, where we have classes **0 (negative)** and **1 (positive)**
- We consider four different types of predictions:
  - **True Positives (TP)**: model predicted 1 correctly
  - **True Negatives (TN)**: model predicted 0 correctly
  - **False Positives (FP)**: it is 0, but model predicted 1
  - **False Negatives (FN)**: it is 1, but model predicted 0
- By using the number of these four types of predictions, we can come up with some useful metrics

Metrics for Classification - The Confusion Matrix

		True condition				
	Total population	Condition positive	Condition negative	Prevalence = $\frac{\Sigma \text{ Condition positive}}{\Sigma \text{ Total population}}$	Accuracy (ACC) = $\frac{\Sigma \text{ True positive} + \Sigma \text{ True negative}}{\Sigma \text{ Total population}}$	
Predicted condition	Predicted condition positive	**True positive**, Power	**False positive**, Type I error	Positive predictive value (PPV), Precision = $\frac{\Sigma \text{ True positive}}{\Sigma \text{ Predicted condition positive}}$	False discovery rate (FDR) = $\frac{\Sigma \text{ False positive}}{\Sigma \text{ Predicted condition positive}}$	
	Predicted condition negative	**False negative**, Type II error	**True negative**	False omission rate (FOR) = $\frac{\Sigma \text{ False negative}}{\Sigma \text{ Predicted condition negative}}$	Negative predictive value (NPV) = $\frac{\Sigma \text{ True negative}}{\Sigma \text{ Predicted condition negative}}$	
		True positive rate (TPR), Recall, Sensitivity, probability of detection $= \frac{\Sigma \text{ True positive}}{\Sigma \text{ Condition positive}}$	False positive rate (FPR), Fall-out, probability of false alarm $= \frac{\Sigma \text{ False positive}}{\Sigma \text{ Condition negative}}$	Positive likelihood ratio (LR+) = $\frac{\text{TPR}}{\text{FPR}}$	Diagnostic odds ratio (DOR) = $\frac{\text{LR+}}{\text{LR−}}$	F_1 score = $\frac{2}{\frac{1}{\text{Recall}} + \frac{1}{\text{Precision}}}$
		False negative rate (FNR), Miss rate $= \frac{\Sigma \text{ False negative}}{\Sigma \text{ Condition positive}}$	True negative rate (TNR), Specificity (SPC) $= \frac{\Sigma \text{ True negative}}{\Sigma \text{ Condition negative}}$	Negative likelihood ratio (LR−) = $\frac{\text{FNR}}{\text{TNR}}$		

Reference: https://en.wikipedia.org/wiki/Confusion_matrix

Metrics for Classification - Precision

- **Precision** aims at answering the following question:
  - **How often the model is correct when it predicts the target should be a certain class?**
- Precision is defined by:

$$Precision = \frac{TP}{TP+FP}$$

- For example:

```
y = (1,0,0,0,1,1,1,0,0)
ŷ = (1,1,1,0,0,1,1,0,0)
```

- We have **TP = 3**, **FP = 2**, so **precision = 3/5 = 60%**

Metrics for Classification - Recall

- **Recall** aims at answering the following question:
  - **How many data of a certain class can the model correctly identifies?**
- Recall is defined by:

$$Recall = \frac{TP}{TP+FN}$$

- For example:

  y = (1,0,0,0,1,1,1,0,0)

  ŷ = (1,1,1,0,0,1,1,0,0)
- We have **TP = 3**, **FN = 1**, so **recall = 3/4 = 75%**

Example: Aeroplane Classifier

- If a model outputs **0** (not-an-aeroplane) for all images:
  - accuracy = 7/9 = 77.7%
  - precision = (undefined)
  - recall = 0%
- If a model outputs **1** (is-an-aeroplane) for all images:
  - accuracy = 2/9 = 22.2%
  - precision = 2/9 = 22.2%
  - recall = 2/2 = 100%

Trade-off Between Precision and Recall

- There is usually a **trade-off** between precision and recall
- For a classifier that outputs a score between 0 and 1, setting the **threshold** has a significant impact on precision and recall
- With a **high** threshold (e.g. 0.8)
  - Most predictions will be considered as **negatives**
  - Precision will tend to be **higher** (model is more **conservative**)
  - Recall will tend to be **lower** (likely to miss some positives)
- With a **lower** threshold (e.g. 0.3)
  - Most predictions will be considered as **positives**
  - Precision will tend to be **lower** (more false positives)
  - Recall will tend to be **higher** (more positives identified)

F-Measure

- **F-Measure** (or **F1 Score**) is a measure that takes both precision and recall into account
- In many cases, we would like to have a **balance** between precision and recall
- F1 Score allows us to choose a model that is reasonable good in terms of both precision and recall
- F1 is the **harmonic average** of precision and recall:

$$F_1 = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}}$$

(Rare Situation) Flaws of F-Measure

- Imagine the dataset is so unbalanced that a model gives:
  - TP = 80, TN = 950, FP = 50, FN = 20
  - Precision = TP / (TP + FP) = 80 / (80 + 50) = 0.6154
  - Recall = TP / (TP + FN) = 80 / (80 + 20) = 0.8
  - F-Measure = 2 / (1/precision + 1/recall) = 0.6957
- When we flip the positive and negative in the above scenario:
  - TN = 80, TP = 950, FN = 50, FP = 20
  - Precision = TP / (TP + FP) = 950 / (950 + 20) = 0.9794
  - Recall = TP / (TP + FN) = 950 / (950 + 50) = 0.95
  - F-Measure = 2 / (1/precision + 1/recall) = 0.8806
- F-Measure is class variant
  - Flipping the case and control really matter

Matthews Correlation Coefficient (MCC)

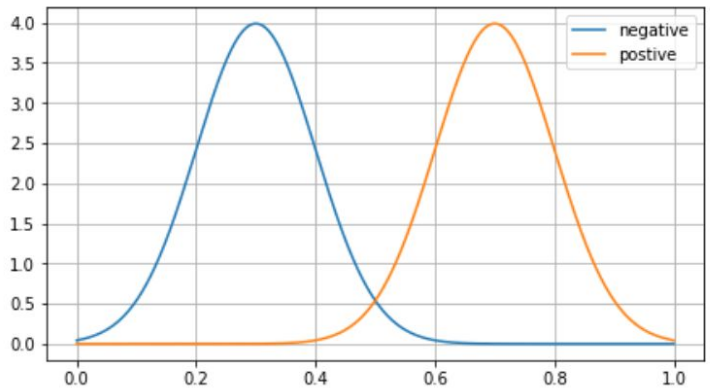
- Advantages of MCC:
  - Class invariant
  - Range between -1 and 1
- Defined as: $$\mathrm{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$
- Using previous example: TP = 80, TN = 950, FP = 50, FN = 20
  - MCC = 0.6679

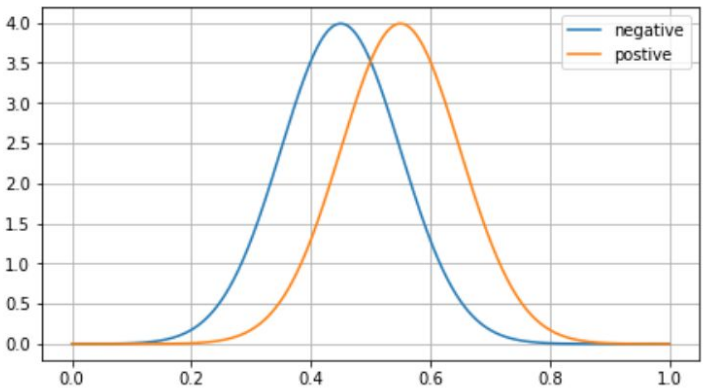
```
from sklearn import metrics
y_test = [1,0,0,0,1,1,1,0,0]
y_pred = [1,1,1,0,0,1,1,0,0]
metrics.matthews_corrcoef(y_test, y_pred)
metrics.f1_score(y_test, y_pred)
```

```
0.35
0.6666666666666666
```

Ranking Positives and Negatives

- In some tasks, we are interested in **accuracy**, ideally we want every prediction to be correct
- However, in some other tasks, it is sufficient that we can identify **one (or a few) positive cases** from a batch of inputs (Examples?)
- Or, in some cases, we are more concerned about the **ranking** of the inputs
- Ideally we want the positives to be always scored **higher** than the negatives





Area Under the ROC Curve (AUC)

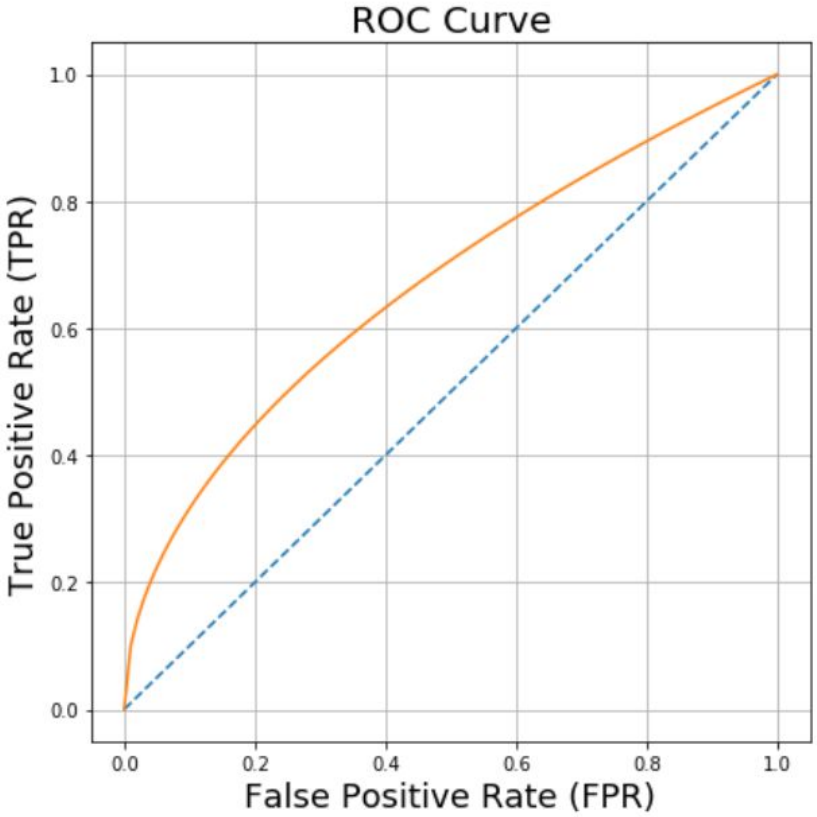
- **AUC** allows us to estimate the ability of a model to rank positives higher than negatives
- The ROC Curve (Receiver Operating Characteristic Curve) is plot by varying the **decision threshold** to obtain different **true positive rates (TPR)** and **false positive rates (FPR)**:

$$TPR = Recall = \frac{TP}{TP+FN}$$

$$FPR = \frac{FP}{FP+TN}$$

Area Under the ROC Curve (AUC)

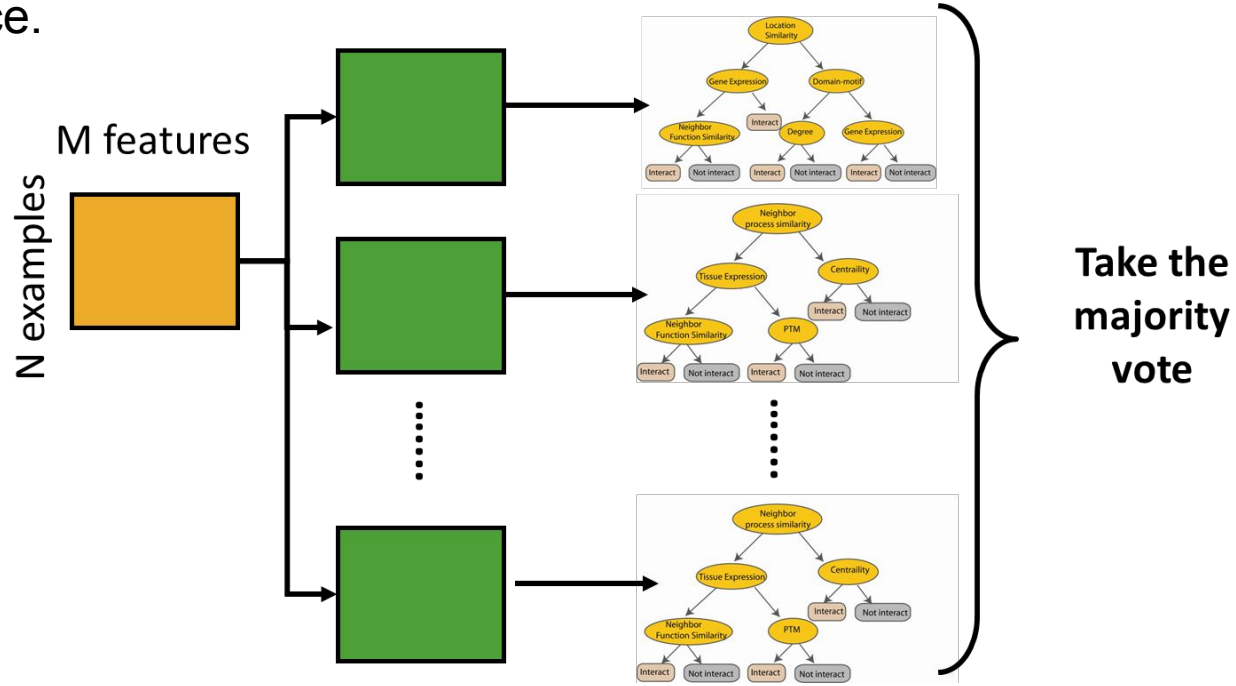
- Get a set of (FPR, TPR) points by varying the decision threshold
- Plotting the points will usually result in a graph that looks like the one on the right
- The larger the **area under the curve**, the better is your model
    - Example: wild guess with 0.5 chance correct, AUC = 0.5



More Classifications

Random Forest

- Random forests are a way of averaging multiple deep decision trees, trained on different parts of the same training set, with the goal of reducing the variance.



Parameters in Random Forest

1. Number of Trees
2. Node size
3. Required split size
4. Maximum number of nodes
5. Maximum depth
6. Quality of a split (e.g. Gini index or entropy)

Steps in Random Forest Algorithm

1. In the Random forest model, a subset of data points and a subset of features is selected for constructing each decision tree. Simply **put n random records and m features** are taken from the data set having k number of records.
2. Individual decision trees are constructed for each sample.
3. Each decision tree will generate an output.
4. Final output is considered based on **Majority Voting** or **Averaging** for Classification and regression, respectively.

Random Forest Example

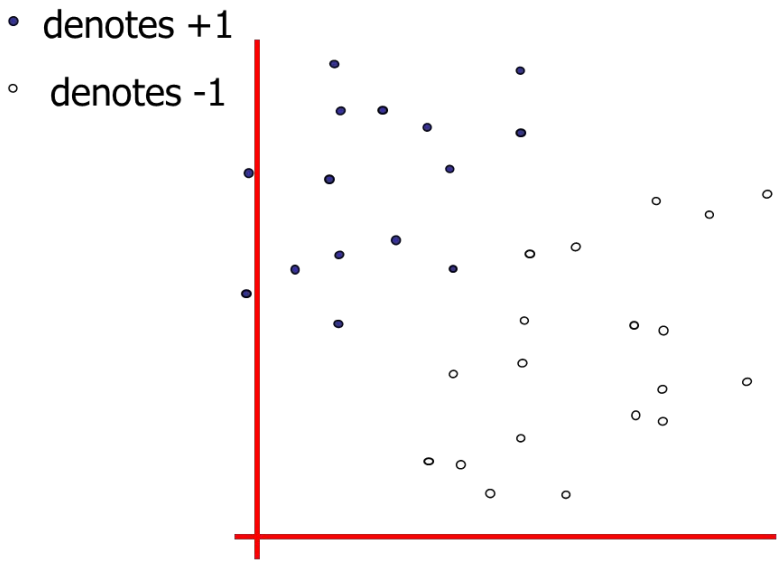
Reference: https://data36.com/random-forest-in-python/

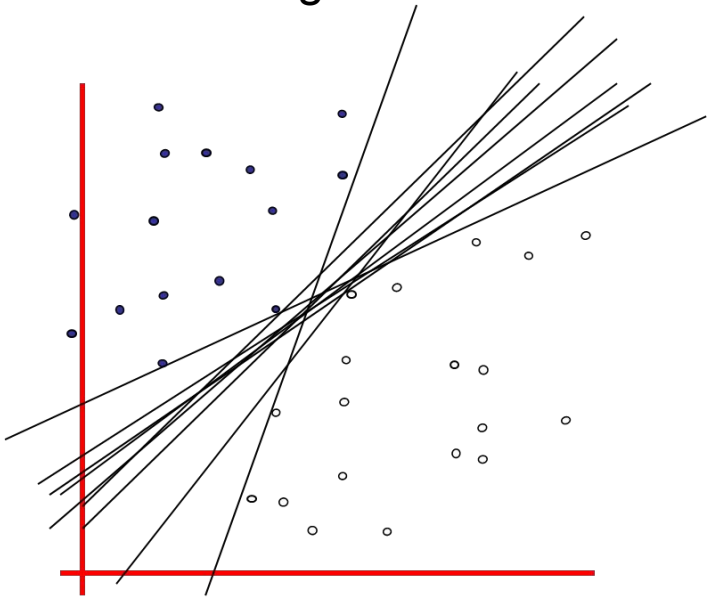
```python
import pandas as pd
df = pd.read_csv("possum.csv")
df.sample(5, random_state=44)
df = df.dropna()
X = df.drop(["case", "site", "Pop", "sex"], axis=1)
y = df["sex"]
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3,random_state=44)
from sklearn.ensemble import RandomForestClassifier
# Changed in version 1.1: The default of max_features changed from "auto" to "sqrt".
rf_model = RandomForestClassifier(n_estimators=50,max_features="sqrt",random_state=44)
rf_model.fit(X_train, y_train)
predictions = rf_model.predict(X_test)
print(predictions)
from sklearn import metrics
print(metrics.accuracy_score(y_test, predictions))
```

```
RandomForestClassifier(n_estimators=50, random_state=44)
['f' 'm' 'm' 'm' 'f' 'f' 'm' 'm' 'm' 'm' 'm' 'f' 'm' 'm' 'm' 'm' 'm' 'f'
 'm' 'm' 'm' 'f' 'f' 'm' 'm' 'm' 'm' 'm' 'm' 'm' 'f']
0.6129032258064516
```

Support Vector Machine (SVM)

- SVM only uses the objects (samples) on the edges of the margin (called support vectors) to separate objects (samples). The separating hyperplane is supported by the vectors (data points) nearest the margin.

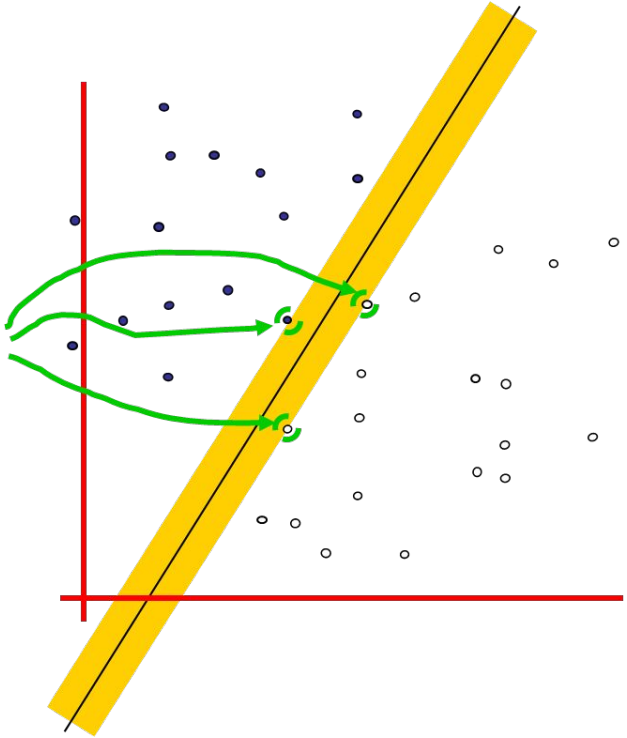




But which one is the best separator?

Support Vector Machine (SVM)

Support Vectors are those data points that the margin pushes up against

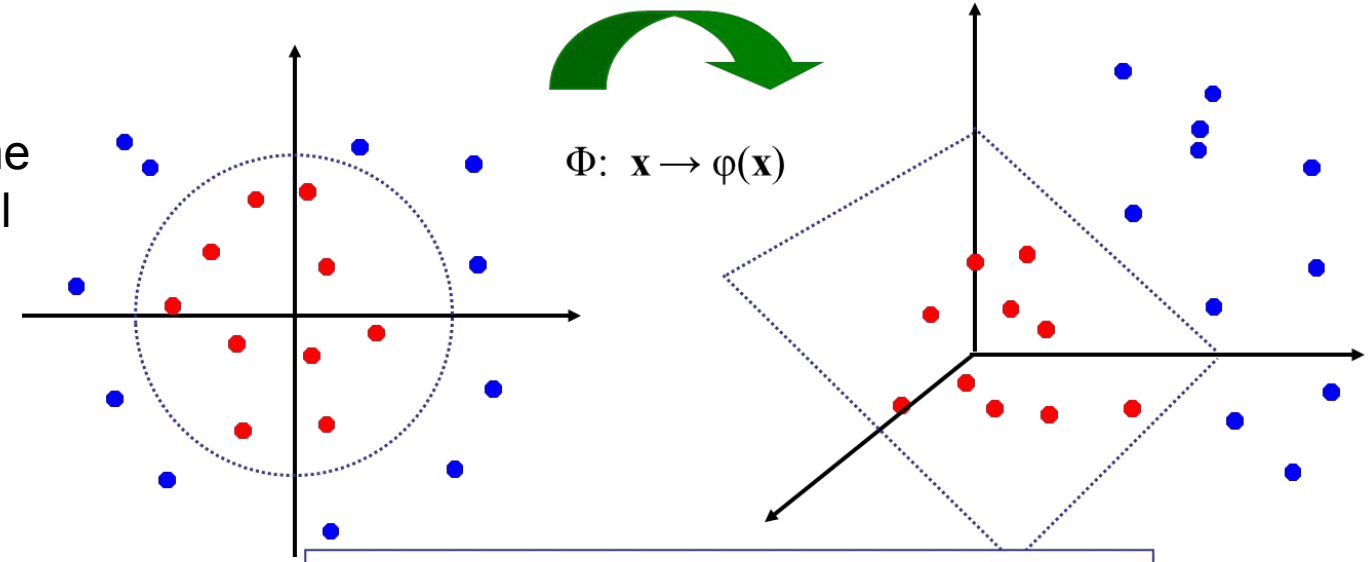


The **maximum margin linear classifier** is the linear classifier with the, um, maximum margin.

This is the simplest kind of SVM (Called an **LSVM**)

Non-linear SVMs

The original input space can always be mapped to some higher-dimensional feature space where the training set is separable



$$\Phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$
$$(x_1, x_2) \mapsto (z_1, z_2, z_3) := (x_1^2, \sqrt{2}\, x_1 x_2, x_2^2)$$

Reference: http://www.cs.cmu.edu/~awm/tutorials

Parameters in SVM

1. **C**: It is the regularization parameter, C, of the error term.
2. **kernel**: It specifies the kernel type to be used in the algorithm.
3. **degree**: It is the degree of the polynomial kernel function ('poly') and is ignored by all other kernels.

Steps in SVM Algorithm

1. Select the appropriate kernel function
2. Define the parameters and constraints
3. Solve the optimization problem to find the optimal hyperplane
4. Make predictions based on the learned model.

SVM: Load Datasets

```python
# Import scikit-learn dataset library
from sklearn import datasets
# Load dataset
cancer = datasets.load_breast_cancer()
# print the names of the 30 features
print("Features: ", cancer.feature_names)
# print the label type of cancer('malignant' 'benign')
print("Labels: ", cancer.target_names)
# print the cancer labels (0:malignant, 1:benign)
print(cancer.target)
```

Reference:
https://www.datacamp.com/tutorial/svm-classification-scikit-learn-python

```
Features:  ['mean radius' 'mean texture' 'mean perimeter' 'mean area' 'mean smoothness'
'mean compactness' 'mean concavity' 'mean concave points' 'mean symmetry' 'mean fractal
dimension' 'radius error' 'texture error' 'perimeter error' 'area error' 'smoothness error'
'compactness error' 'concavity error' 'concave points error' 'symmetry error' 'fractal
dimension error' 'worst radius' 'worst texture' 'worst perimeter' 'worst area' 'worst
smoothness' 'worst compactness' 'worst concavity' 'worst concave points' 'worst symmetry'
'worst fractal dimension']
Labels:  ['malignant' 'benign']
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 ......
 1 1 1 1 1 1 1 0 0 0 0 0 0 1]
```

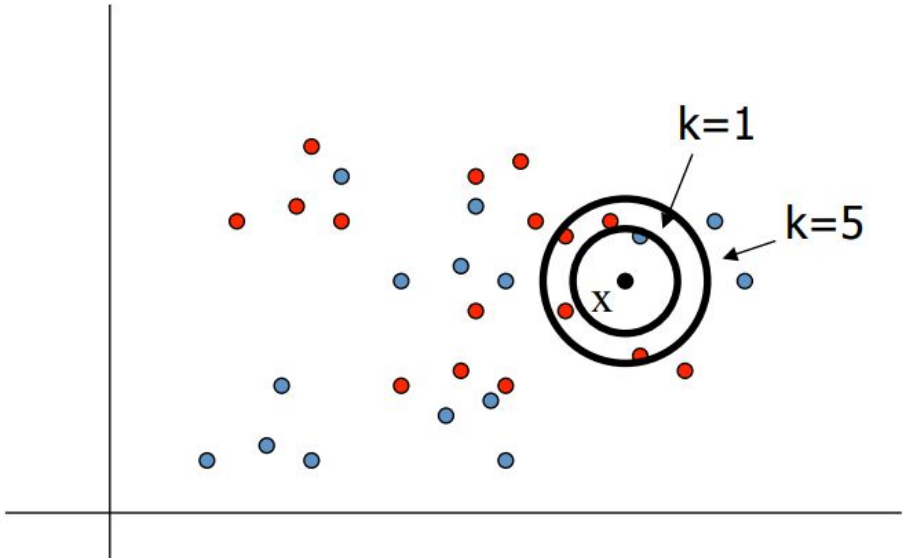
SVM: Preprocess Data, Train and Use Model

```
# Import train_test_split function
from sklearn.model_selection import train_test_split
# Split dataset into training set and test set
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
test_size=0.3,random_state=109) # 70% training and 30% test
# Import svm model
from sklearn import svm
# Create a svm Classifier
clf = svm.SVC(kernel='linear') # Linear Kernel
# Train the model using the training sets
clf.fit(X_train, y_train)
# Predict the response for test dataset
y_pred = clf.predict(X_test)
# Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics
# Model Accuracy: how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

```
SVC(kernel='linear')
Accuracy: 0.9649122807017544
```

K-Nearest Neighbour (K-NN)

- To classify a new input vector x, examine the k-closest training data points to x and assign the object to the most frequently occurring class
- For example,
  - If k = 1, pred(x) = **blue**
  - If k = 5, pred(x) = **red**
- Parameter:
  - n_neighbor: (default 5)
  - weights: (default: “uniform“)



Reference: http://people.csail.mit.edu/dsontag/courses/ml12/slides/lecture10.pdf

Steps in K-NN Algorithm

1. Select the number K of the neighbors.
2. Calculate the Euclidean distance of K number of neighbors.
3. Take the K nearest neighbors as per the calculated Euclidean distance.
4. Among these k neighbors, count the number of the data points in each category.

kNN: Load Data and Preprocess Data

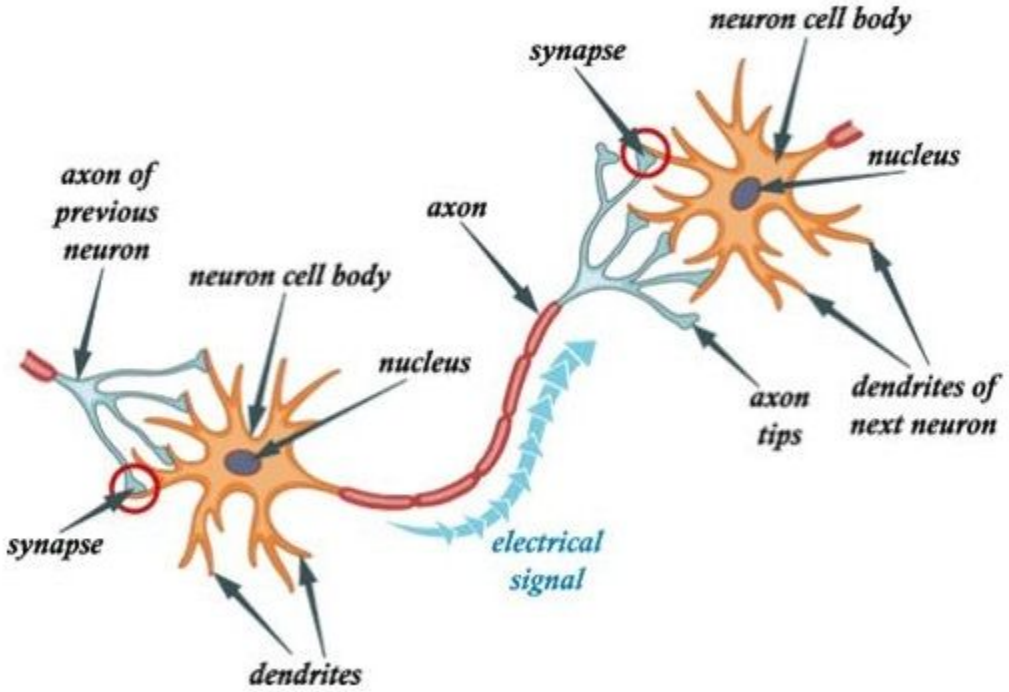
```python
# Import necessary modules
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
# Loading data
irisData = load_iris()
# Create feature and target arrays
X = irisData.data
y = irisData.target
# Split into training and test set
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size = 0.2,random_state=42)
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)
# Predict on dataset which model has not seen before
print(knn.predict(X_test))
# Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics
# Model Accuracy: how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, knn.predict(X_test)))
```

Reference:
https://www.geeksforgeeks.org/k-nearest-neighbor-algorithm-in-python/

```
KNeighborsClassifier(n_neighbors=7)
[1 0 2 1 1 0 1 2 2 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 0.9666666666666667
```

Neural Network (NN) / Artificial Neural Network (ANN)

- A computational model inspired by the biological neural networks in the human body
- A model involving a set of **neurons** connected to one another in some specific ways
- Can be used to model **non-linear** relations between inputs and outputs



Simple ANN: Initialize ANN

```python
import torch
import torch.nn as nn
# input size, hidden unit, output size, batch size, and the learning rate.
n_input, n_hidden, n_out, batch_size, learning_rate = 10, 15, 1, 100, 0.01
# randomly initialize the dummy input and the output target data (or tensor)
data_x = torch.randn(batch_size, n_input)
data_y = (torch.rand(size=(batch_size, 1)) < 0.5).float()
# define ANN model
model = nn.Sequential(nn.Linear(n_input, n_hidden),nn.ReLU(),
                      nn.Linear(n_hidden, n_out),nn.Sigmoid())
print(model)
loss_function = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

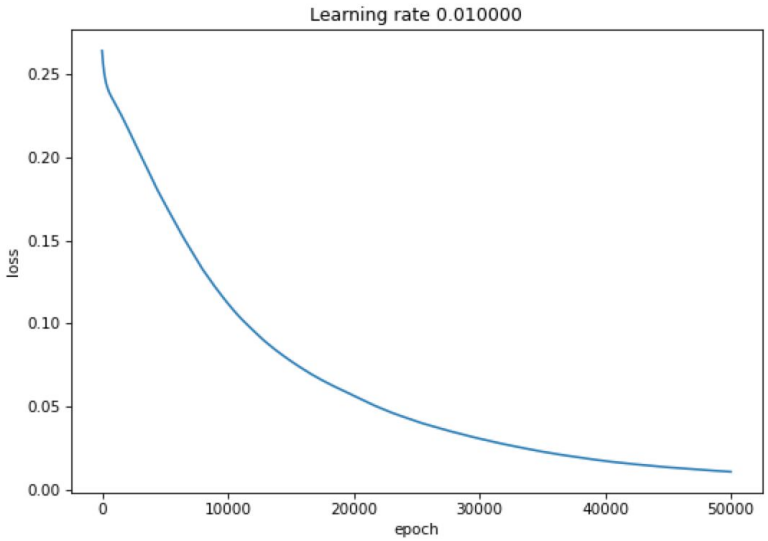
```
Sequential(
  (0): Linear(in_features=10, out_features=15, bias=True)
  (1): ReLU()
  (2): Linear(in_features=15, out_features=1, bias=True)
  (3): Sigmoid()
)
```

Reference:
https://towardsdatascience.com/build-a-simple-neural-network-using-pytorch-38c55158028d

Simple ANN: Train the Model, Visualize Training Performance

```
# Training Loop
losses = []
for epoch in range(50000):
    pred_y = model(data_x)  # Forward propagation
    loss = loss_function(pred_y, data_y)
    losses.append(loss.item())
    model.zero_grad()
    loss.backward()         # Backward propagation
    optimizer.step()        # Gradient descent

import matplotlib.pyplot as plt
plt.plot(losses)
plt.ylabel('loss')
plt.xlabel('epoch')
plt.title("Learning rate %f" % (learning_rate))
plt.show()
```

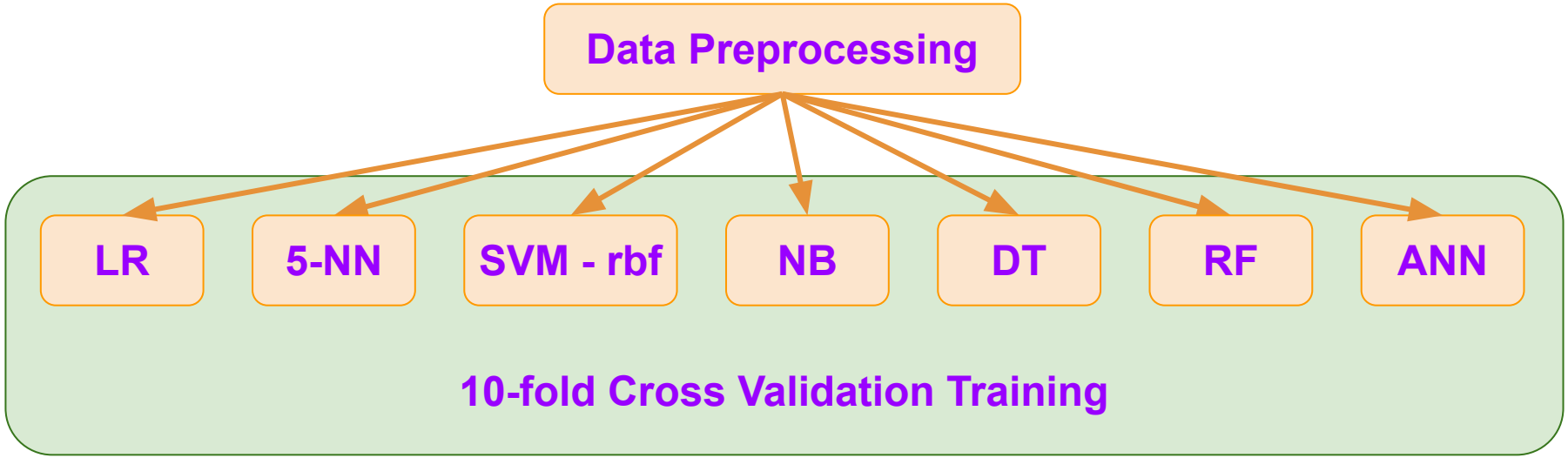



```
[<matplotlib.lines.Line2D object at 0x0000013226A89AD0>]
Text(0, 0.5, 'loss')
Text(0.5, 0, 'epoch')
Text(0.5, 1.0, 'Learning rate 0.010000')
```

Model Comparisons, and Parameter Tuning

Comparisons between Different Algorithms

- Goal: A fair comparison between different classification algorithms



Comparisons between Different Algorithms

- Problem: Predict (classify) whether the customer will purchase the iPhone or not based on gender, age and salary.

Data:

	Gender	Age	Salary	Purchased Iphone
2	Male	19	19000	0
3	Male	35	20000	0
4	Female	26	43000	0
5	Female	27	57000	0
6	Male	19	76000	0
7	Male	27	58000	0
8	Female	27	84000	0
9	Female	32	150000	1
10	Male	25	33000	0
11	Female	35	65000	0

400 customers x [3 features + 1 class label]

Reference:
https://towardsdatascience.com/machine-learning-project-17-compare-classification-algorithms-87cb50e1cb60

Comparisons between Different Algorithms

```python
# Step 1 - Load Data
import pandas as pd
dataset = pd.read_csv("iphone_purchase_records.csv")
X = dataset.iloc[:,:-1].values
y = dataset.iloc[:, 3].values

# Step 2 - Convert Gender to number
from sklearn.preprocessing import LabelEncoder
labelEncoder_gender =  LabelEncoder()
X[:,0] = labelEncoder_gender.fit_transform(X[:,0])

# Step 3 - Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X = sc.fit_transform(X)

# Step 4 - Load Classification Algorithm Libraries
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
# continue to next page
```

Comparisons between Different Algorithms

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

# Step 5 - Compare Classification Algorithms
classification_models = []
classification_models.append(('Logistic Regression', LogisticRegression(solver="liblinear")))
classification_models.append(('K Nearest Neighbor', KNeighborsClassifier(n_neighbors=5, metric="minkowski",p=2)))
classification_models.append(('Kernel SVM', SVC(kernel = 'rbf',gamma='scale')))
classification_models.append(('Naive Bayes', GaussianNB()))
classification_models.append(('Decision Tree', DecisionTreeClassifier(criterion = "entropy")))
classification_models.append(('Random Forest', RandomForestClassifier(n_estimators=100, criterion="entropy")))
# continue to next page
```

Comparisons between Different Algorithms

```
for name, model in classification_models:
  kfold = KFold(n_splits=10)
  result = cross_val_score(model, X, y, cv=kfold, scoring='accuracy')
  print("%s: Mean Accuracy = %.2f%% - SD Accuracy = %.2f%%" % (name, result.mean()*100, result.std()*100))

# now, ANN
import torch
import torch.nn as nn
from sklearn import metrics
n_input, n_hidden, n_out, batch_size, learning_rate = X.shape[1], 15, 1, 10, 0.01
results = []

for fold in np.arange(10):
  X_train = []
  X_test = []
  y_train  = []
  y_test = []
# continue to next page
```

Comparisons between Different Algorithms

```
  for i in np.arange(X.shape[0]):
    if i % 10 != fold:
      X_train.append(X[i])
      y_train.append([y[i]])
    else:
      X_test.append(X[i])
      y_test.append([y[i]])

  X_train = np.array(X_train)
  y_train = np.float64(np.array(y_train))
  X_test = np.array(X_test)

  model = nn.Sequential(nn.Linear(n_input, n_hidden), nn.ReLU(),
    nn.Linear(n_hidden, n_out), nn.Sigmoid())
  loss_function = nn.MSELoss()
  optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
  losses = []

# continue to next page
```

Comparisons between Different Algorithms

```
  for epoch in range(5000):
    pred_y = model(torch.FloatTensor(X_train))
    loss = loss_function(pred_y, torch.FloatTensor(y_train))
    losses.append(loss.item())
    model.zero_grad()
    loss.backward()
    optimizer.step()

  targets = model(torch.FloatTensor(X_test))
  results.append(metrics.accuracy_score(np.round(targets.detach().numpy()), y_test))

print("ANN: Mean Accuracy = %.2f%% - SD Accuracy = %.2f%%" % (np.mean(results)*100,
np.std(results)*100))
```

```
Logistic Regression: Mean Accuracy = 82.75% - SD Accuracy = 11.37%
K Nearest Neighbor: Mean Accuracy = 90.50% - SD Accuracy = 7.73%
Kernel SVM: Mean Accuracy = 90.75% - SD Accuracy = 9.15%
Naive Bayes: Mean Accuracy = 85.25% - SD Accuracy = 10.34%
Decision Tree: Mean Accuracy = 84.75% - SD Accuracy = 8.10%
Random Forest: Mean Accuracy = 88.50% - SD Accuracy = 8.96%
ANN: Mean Accuracy = 89.25% - SD Accuracy = 2.51%
```

Note: This is not the cross validation accuracy.

Notes on Tuning Parameter, Hyperparameters

- Parameters are the configuration model, which are internal to the model.
  - The coefficients (or weights) of linear and logistic regression models.
  - Weights and biases of a nn
  - The cluster centroids in clustering
- Hyperparameters are the explicitly specified parameters that control the training process.
  - Train-test split ratio
  - Learning rate in optimization algorithms (e.g. gradient descent)
  - Choice of optimization algorithm (e.g., gradient descent, stochastic gradient descent, or Adam optimizer)
  - Choice of activation function in a neural network (nn) layer (e.g. Sigmoid, ReLU, Tanh)
  - The choice of cost or loss function the model will use

Notes on Tuning Parameter, Hyperparameters

- Number of hidden layers in a nn
- Number of activation units in each layer
- The drop-out rate in nn (dropout probability)
- Number of iterations (epochs) in training a nn
- Number of clusters in a clustering task
- Kernel or filter size in convolutional layers
- Pooling size
- Batch size

- We need a clear understanding of what parameters and hyperparameters are before the turning. Others may challenge you as reporting the best trial based on different settings (See multiple comparisons problem).
- Never simply report the result based on the best set of parameters. If you don't know how to pick, just **use the default**.

Good Examples for Turning Hyperparameters

1. **Grid Search**:
   - Define a set of hyperparameter values to search over
   - Try all possible combinations of these values
   - Report the results with multiple hypotheses correction
2. **Manual Search by Expert Knowledge**:
   - Explain the set of hyperparameters used based on the problem specific domain
3. **Random Search**:
   - Select a combination of hyperparameters from a predefined set
   - Only train the models using those sets
   - Report the results with multiple hypotheses correction

See more: https://www.run.ai/guides/hyperparameter-tuning

Regression

Regression

- Regression Analysis measures the nature and extent of the relationship between two or more variables, thus, enables us to make predictions.
- Regression is the measure of the average relationship between two or more variables.
- The target is also known as dependent variable.
- The predictors are known as independent variables.
- The regression line shows the average relations between two variables. It is also called as **line of the best fit**.
- In most regression problems, the **target** is a **continuous** variable.

Some Top Regression Algorithms

1. (Multiple) Linear Regression
2. Logistic Regression
3. Ridge Regression
4. Lasso Regression
5. Neural Network Regression
6. Support Vector Regressor

Linear Regression

- In regression, we want to predict the values of some targets given some **predictors** or **features**
- In the simplest case, we can assume that the relationship between the features and the target is **linear**:

$$y = a + bX$$

- In the equation above, y is the **target**, X is the **feature**, a is the **intercept**, and b is the **weight** of the feature
- **Learning** means estimating a suitable value for both a and b such that the resultant straight line best approximate the data we observed

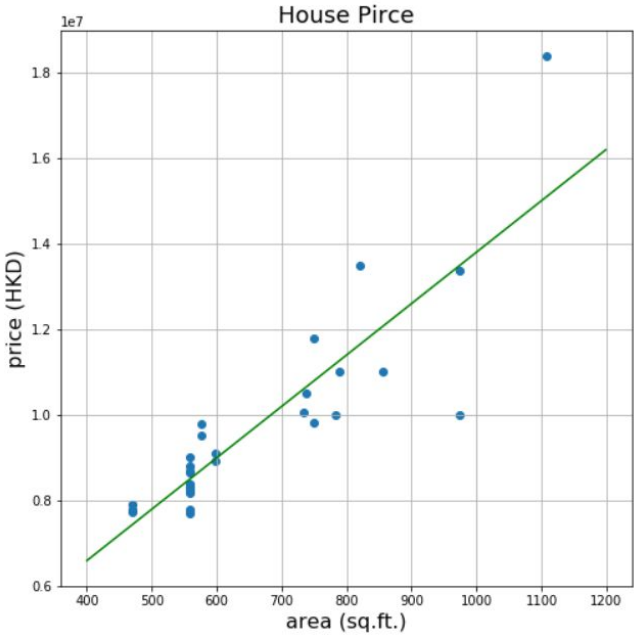
Linear Regression

- Consider an example of predicting the **price of a house** given its **area**
- A few samples from a house price dataset

	area (sq.ft.)	price (HKD)
0	597	9100000
1	469	7725000
2	558	8200000
3	558	8350000
4	558	8650000

Linear Regression

- Using ordinary least squares method, we can estimate a and b in the equation
- The line with the estimated values of a and b is plotted on the right
- Note that **not** all points are lying on the line (Why?)
- How can we **predict** the price of other houses?
- How do we tell whether this is a **good** model?



Linear Regression

- In the above example, we have **one** feature, which is the area of the house
- Can you think of **other features** that are useful for predicting the price?
- If we have more features, the equation will become

$$y = b_0 + b_1x_1 + b_2x_2 + \ldots + b_nx_n$$

- This is still a **linear regression** model, sometimes called **multiple linear regression**
- b_0 is called the **bias term**, while b_1 to b_n are the weights of the features
- y is a **weighted linear combination** of the values of x_1 to x_n

Linear Regression

- If we consider $x_0 = 1$, then we can write the linear regression equation in terms of vectors
- Let x be the feature vector $(x_0, x_1, \ldots, x_n)$ representing the input
- Let b as a vector of the weights $b_0, b_1, \ldots, b_n$
  (The b_i's are the parameters of this model, and in many cases are denoted by θ)

$$y = \theta^T x$$

- If we consider the **whole training dataset** with m samples, and each input has n features (and thus we will have $n+1$ parameters):

$$(y_1, y_2, \cdots, y_m) = (b_0, b_1, \cdots, b_2) \begin{pmatrix} x_{1,0} & x_{1,1} & \cdots & x_{1,n}) \\ x_{2,0} & x_{2,1} & \cdots & x_{2,n}) \\ \vdots & \vdots & \ddots & \vdots) \\ x_{m,0} & x_{m,1} & \cdots & x_{m,n} \end{pmatrix}^T$$

Short Summary

1. What is a **model**?
   - Your assumptions of how things work
   - A simplification of the world (or at least the problem you are trying to solve)
   - A model is usually characterized by some **parameters**
   - (Thus machine learning usually involves **parameter optimization**)
2. The inputs are **represented** by **feature vectors**
   - **Representation** of input data is very important in machine learning
   - Very often we may even want to learn how to represent the raw data

Example: Linear Regression

- To predict the stock index price based on interest rate and unemployment rate using multiple linear model
- Based on the model, evaluate the coefficient of determination

Year	Month	Interest_Rate	Unemployment_Rate	Stock_Index_Price
2017	12	2.75	5.3	1464
2017	11	2.5	5.3	1394
2017	10	2.5	5.3	1357
2017	9	2.5	5.3	1293
2017	8	2.5	5.4	1256
2017	7	2.5	5.6	1254
2017	6	2.5	5.5	1234
2017	5	2.25	5.5	1195
2017	4	2.25	5.5	1159
2017	3	2.25	5.6	1167
2017	2	2	5.7	1130
2017	1	2	5.9	1075
2016	12	2	6	1047
2016	11	1.75	5.9	965
2016	10	1.75	5.8	943
2016	9	1.75	6.1	958
2016	8	1.75	6.2	971
2016	7	1.75	6.1	949
2016	6	1.75	6.1	884
2016	5	1.75	6.1	866
2016	4	1.75	5.9	876
2016	3	1.75	6.2	822
2016	2	1.75	6.2	704
2016	1	1.75	6.1	719

Reference: https://datatofish.com/multiple-linear-regression-python/

Example: LR - Generate Synthetic Dataset

```
import pandas as pd
Stock_Market = {'Year':
[2017,2017,2017,2017,2017,2017,2017,2017,2017,2017,2017,2017,2016,2016,2016,2016,2016,2
016,2016,2016,2016,2016,2016,2016],
'Month': [12, 11,10,9,8,7,6,5,4,3,2,1,12,11,10,9,8,7,6,5,4,3,2,1],
'Interest_Rate':
[2.75,2.5,2.5,2.5,2.5,2.5,2.5,2.25,2.25,2.25,2,2,2,1.75,1.75,1.75,1.75,1.75,1.75,1.75,1
.75,1.75,1.75,1.75],
'Unemployment_Rate':
[5.3,5.3,5.3,5.3,5.4,5.6,5.5,5.5,5.5,5.6,5.7,5.9,6,5.9,5.8,6.1,6.2,6.1,6.1,6.1,5.9,6.2,
6.2,6.1],
'Stock_Index_Price':
[1464,1394,1357,1293,1256,1254,1234,1195,1159,1167,1130,1075,1047,965,943,958,971,949,8
84,866,876,822,704,719]}
df =
pd.DataFrame(Stock_Market,columns=['Year','Month','Interest_Rate','Unemployment_Rate','
Stock_Index_Price'])
```

Example: LR - Train and Use Model

```python
from sklearn import linear_model
import statsmodels.api as sm
# We have 2 variables for multiple regression
X = df[['Interest_Rate','Unemployment_Rate']]
Y = df['Stock_Index_Price']
regr = linear_model.LinearRegression()
regr.fit(X, Y)
print('Intercept: ', regr.intercept_)  # 1798.403977625855
print('Coefficients: ', regr.coef_)    # [345.54008701 -250.14657137]
# MLR: Y = 345X0 - 250X1 + 1798
# prediction with sklearn
New_Interest_Rate = 2.75
New_Unemployment_Rate = 5.3
print('Predicted Stock Index Price: ', regr.predict([[New_Interest_Rate
,New_Unemployment_Rate]]))
```

```
LinearRegression()
Intercept: 1798.403977625855
Coefficients: [ 345.54008701 -250.14657137]
Predicted Stock Index Price: [1422.86238865]
```

Example: LR - R^2

```
r2 = regr.score(X, Y)                                          # 0.8976335894170215
adjusted_r2 = 1 - (1-r2)*(len(Y)-1)/(len(Y)-X.shape[1]-1) # 0.887884407456738
print(r2,' ',adjusted_r2)
```

```
0.8976335894170215   0.8878844074567379
```

Note: There is a linearity between X and Y

Metrics: Coefficient of Determination R^2

- Assesses how strong the linear relationship is between the variance in the dependent variables and prediction from the independent variables
- Some notations:
  - Observed data is y_i, while mean of observed data is $\bar{y}$
- Formulae:
  - Total sum of squares: $SS_{tot} = \sum_i (y_i - \bar{y})^2$
  - Sum of squares of residuals: $SS_{res} = \sum_i (y_i - f_i)^2$
  - Coefficient of determination: $R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum_i (y_i - f_i)^2}{\sum_i (y_i - \bar{y})^2}$
- R^2 gives some information about the goodness-of-fit of a model, $0 \leq R^2 \leq 1$
  - $R^2 = 1$ means prediction perfectly fit the data

Metrics: Adjusted Coefficient of Determination R^2(adj)

- Problem: The value of the coefficient of determination can never decrease with the addition of more variables into the regression model. Hence, R^2 can be artificially inflated as more variables are included in the model.
- The adjusted R^2 value represents the percentage of variation in the response variable explained by the independent variables, corrected for degrees of freedom.

$$R^2(adj) = 1 - \frac{(n-1)(1-R^2)}{n-k-1}$$

k: number of independent variables in the model, excluding constant

Example: LR - Comprehensive Statistics Info

```
# with statsmodels, adding a constant
X = sm.add_constant(X)
# ordinary least squares
model = sm.OLS(Y, X).fit()
predictions = model.predict(X)
print_model = model.summary()
print(print_model)
```

Example: LR - Comprehensive Statistics Info

```
                            OLS Regression Results
==============================================================================
Dep. Variable:      Stock_Index_Price   R-squared:                       0.898
Model:                            OLS   Adj. R-squared:                  0.888
Method:                 Least Squares   F-statistic:                     92.07
Date:                Sun, 08 Oct 2023   Prob (F-statistic):           4.04e-11
Time:                        22:35:31   Log-Likelihood:                -134.61
No. Observations:                  24   AIC:                             275.2
Df Residuals:                      21   BIC:                             278.8
Df Model:                           2
Covariance Type:            nonrobust
=====================================================================================
                        coef    std err          t      P>|t|      [0.025      0.975]
-------------------------------------------------------------------------------------
const              1798.4040    899.248      2.000      0.059     -71.685    3668.493
Interest_Rate       345.5401    111.367      3.103      0.005     113.940     577.140
Unemployment_Rate  -250.1466    117.950     -2.121      0.046    -495.437      -4.856
==============================================================================
Omnibus:                        2.691   Durbin-Watson:                   0.530
Prob(Omnibus):                  0.260   Jarque-Bera (JB):                1.551
Skew:                          -0.612   Prob(JB):                        0.461
Kurtosis:                       3.226   Cond. No.                         394.
==============================================================================
```

More about Prediction

1. **Relevant data range**
   - Model can only predict within the relevant range of data
2. **Confidence interval for predicted Y given X**
   - According the indirect estimation in variance reduction technique, mean and variance of prediction should be calculated based on X instead of prediction
3. **Correlation analysis**
   - Correlation analysis is used to measure the strength of the linear relationship between Y and X_i

Metrics: MAE

- In regression tasks, for each test data point, we have a ground truth value y and a predicted value $\hat{y}$
- Intuitively, a model is good if all $\hat{y}$'s are close to their respective y's
- Hence, a straight forward metric would be the **Mean Absolute Error (MAE)**:
  - The mean of the absolute differences for all samples

$$MAE(y, \hat{y}) = \frac{1}{n} \sum_{i=0}^{n-1} |y_i - \hat{y}_i|$$

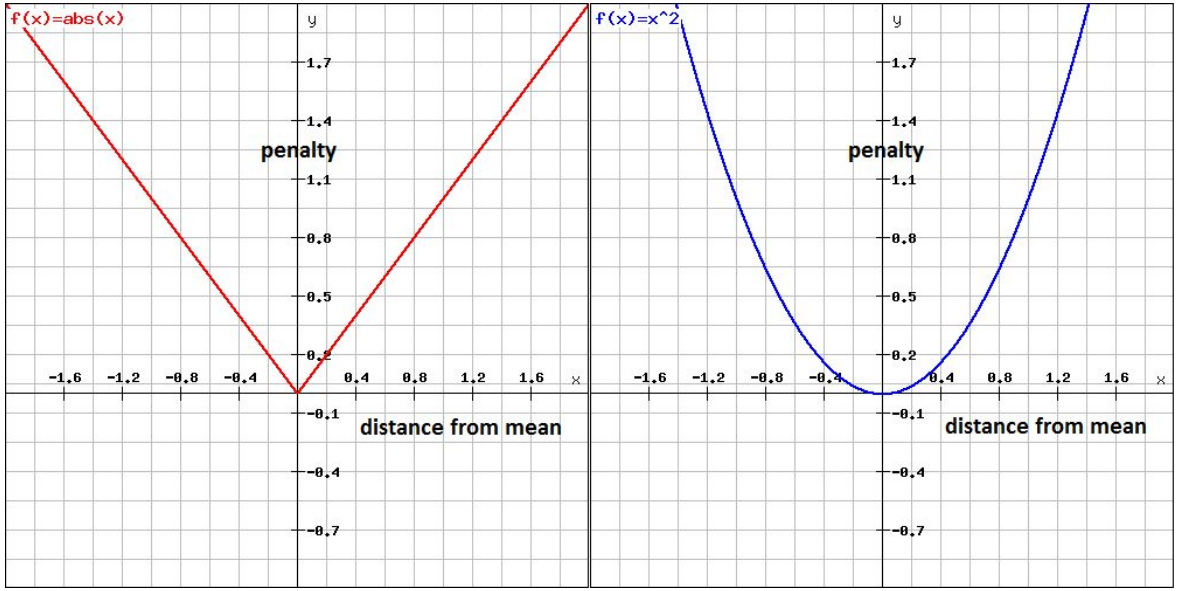
Metrics: RMSE

- Another commonly used metric for regression is the **Root Mean Squared Error (RMSE)**
  - The square root of the mean of squared differences for all samples
  - Let y be the vector of ground truth, and $\hat{\mathrm{y}}$ be the vector of predicted values, root mean squared error is defined by:

$$RMSE(y, \hat{y}) = \sqrt{\frac{1}{n}\sum_{i=0}^{n-1}(y_i - \hat{y}_i)^2}$$

Metrics for Regression

- MSE will be **larger** if the model makes predictions that are **far** from the truth values (due to the square)



Reference:
https://stats.stackexchange.com/questions/147001/is-minimizing-squared-error-equivalent-to-minimizing-absolute-error-why-squared

Logistic Regression

- We can apply linear regression on this problem, but it is likely to perform badly
- For **classification**, we want to have a **decision rule**, linear regression output lies on a straight line, and may not allow picking a threshold easily
- We don't really want the model to output some values too different from 0 or 1
- However, we can apply a **transformation** to the output of linear regression:

$$y = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

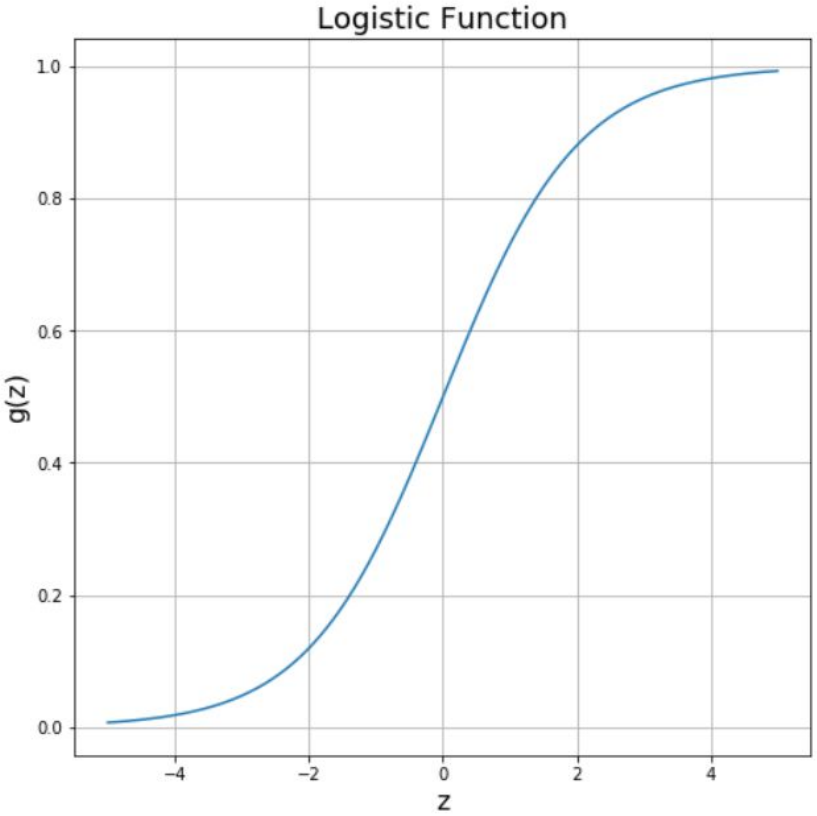
- $g(z)$ is called the **logistic function** or **sigmoid function**

Logistic Regression

$$y = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

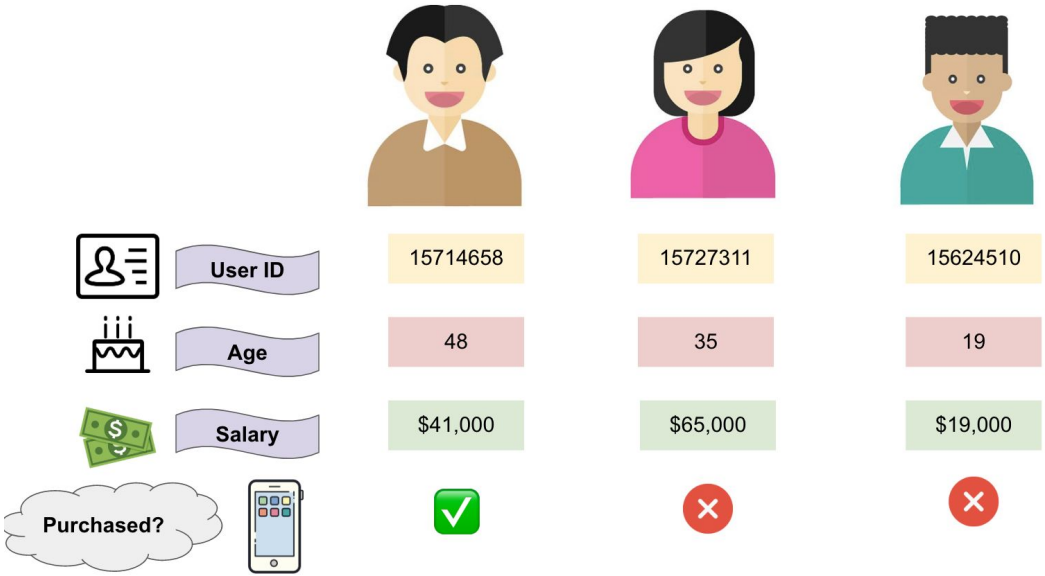
- The logistic function has certain characteristics

1. Its value is always bounded by 0 and 1
2. Its value tends to 1 if z tends to +∞, and tends to 0 if z tends to −∞
3. It is more likely to output values close to 0 or 1



Example: Logistic Regression

- The client information include estimated salary, gender, age and customer ID. To create a logistic regression model whether customer will purchase an iPhone or not




```
>>> X_train
     Age  EstimatedSalary  Gender_Female  Gender_Male
250   44            39000              1            0
63    32           120000              0            1
312   38            50000              1            0
159   32           135000              1            0
283   52            21000              1            0
..   ...              ...            ...          ...
323   48            30000              1            0
192   29            43000              0            1
117   36            52000              0            1
47    27            54000              1            0
172   26           118000              1            0

[300 rows x 4 columns]
```

```
>>> X_train
array([[ 0.58164944, -0.88670699,  0.98019606, -0.98019606],
       [-0.60673761,  1.46173768, -1.02020406,  1.02020406],
       [-0.01254409, -0.5677824 ,  0.98019606, -0.98019606],
       ...,
       [-0.21060859, -0.50979612, -1.02020406,  1.02020406],
       [-1.10189888, -0.45180983,  0.98019606, -0.98019606],
       [-1.20093113,  1.40375139,  0.98019606, -0.98019606]])
```

Reference: https://www.vebuso.com/2020/02/linear-to-logistic-regression-explained-step-by-step/

Example: Logistic Regression - Preprocess Data

```python
import numpy as np
import pandas as pd
# import the dataset
dataset = pd.read_csv('Social_Network_Ads.csv')
# get dummy variables
df_getdummy=pd.get_dummies(data=dataset, columns=['Gender'])
# separate X and Y variables
X = df_getdummy.drop(['User ID', 'Purchased'],axis=1)
Y = df_getdummy['Purchased']
# split the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train,X_test,Y_train,Y_test = train_test_split(X, Y, test_size=0.25, random_state=0)
# feature scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Example: Logistic Regression - Train LR model

```python
# fit Logistic Regression to the training set
from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression(random_state = 0)
classifier.fit(X_train, Y_train)
print('Intercept: ', classifier.intercept_)    # -0.96134888
print('Coefficients: ', classifier.coef_)
```

```
LogisticRegression(random_state=0)
Intercept: [-0.96134888]
Coefficients: [[ 2.09617526  1.11788402 -0.04789188  0.04789188]]
```

Trained logistic regression model:

$$Y = f(X) = \frac{e^{-0.96+2.10x_0+1.12x_1-0.05x_2+0.05x_3}}{1+e^{-0.96+2.10x_0+1.12x_1-0.05x_2+0.05x_3}}$$

Example: Logistic Regression - Prediction and Confusion Matrix

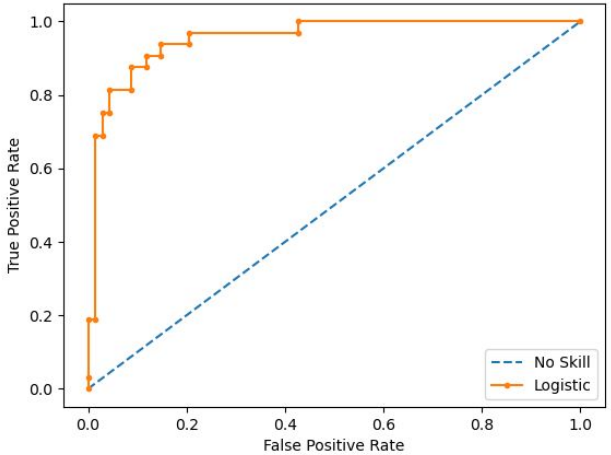
- Based on confusion matrix, we can plot ROC or PRC curves
- We can also evaluate comprehensive statistics information similar to previous example

```python
# predict the test set results
Y_pred = classifier.predict(X_test)
# make the confusion matrix for evaluation
from sklearn.metrics import confusion_matrix
m = confusion_matrix(Y_test, Y_pred)
print(m) # [[65, 3], [6, 26]]
```

```
array([[65,  3],
       [ 6, 26]], dtype=int64)
```

Example: Logistic Regression - ROC Curve

```
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from matplotlib import pyplot
# generate a no skill prediction (majority class)
ns_probs = [0 for _ in range(len(Y_test))]
# keep probabilities for the positive outcome only
lr_probs = classifier.predict_proba(X_test)[:, 1]
# calculate scores
ns_auc = roc_auc_score(Y_test, ns_probs)
lr_auc = roc_auc_score(Y_test, lr_probs)
# summarize scores
print('No Skill: ROC AUC=%.3f' % (ns_auc)) # No Skill: ROC AUC=0.500
print('Logistic: ROC AUC=%.3f' % (lr_auc)) # Logistic: ROC AUC=0.955
ns_fpr, ns_tpr, _ = roc_curve(Y_test, ns_probs) # calculate roc curves
lr_fpr, lr_tpr, _ = roc_curve(Y_test, lr_probs) # calculate roc curves
# plot the roc curve for the model
pyplot.plot(ns_fpr, ns_tpr, linestyle='--', label='No Skill')
pyplot.plot(lr_fpr, lr_tpr, marker='.', label='Logistic')
pyplot.xlabel('False Positive Rate')  # axis labels
pyplot.ylabel('True Positive Rate')   # axis labels
pyplot.legend()                       # show the legend
pyplot.show()                         # show the plot
```



Reference:
https://machinelearningmastery.com/roc-curves-and-precision-recall-curves-for-classification-in-python/

There are many many different regression models

1. Lasso regression
   - $\min\left(\sum \epsilon_i^2 + \lambda \sum |\beta_i|\right) = \min \sum (y - \beta_0 - \sum \beta_i x_i)^2 + \lambda \sum |\beta_i|$
2. Ridge regression
   - $\min\left(\sum \epsilon_i^2 + \lambda \sum \beta_i^2\right) = \min \sum (y - \beta_0 - \sum \beta_i x_i)^2 + \lambda \sum \beta_i^2$
3. Principal components regression
   - $U_1 = \sum \beta_i x_i \text{such that} \sum \beta_i^2 = 1$ is the first principal component
4. Support vector regressor
   - Use non-linear kernel functions to find the optimal solutions for non-linear models
5. … etc

Reference: https://www.listendata.com/2018/03/regression-analysis.html

Example: Ridge Regression

Reference:
https://vitalflux.com/ridge-regression-concepts-python-example/

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score
from sklearn import datasets
boston_ds = datasets.load_boston()       # Load the Sklearn Boston Dataset
X = boston_ds.data
y = boston_ds.target
X_train,X_test,y_train,y_test = train_test_split(X, y, test_size=0.3, random_state=42)
# Fit a pipeline using Training dataset and related labels
pipeline = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
pipeline.fit(X_train, y_train)
y_train_pred = pipeline.predict(X_train) # Predicted value for training set
y_test_pred = pipeline.predict(X_test)   # Predicted value for testing set
print('MSE train: %.3f, test: %.3f\nR^2 train: %.3f, test: %.3f' %
(mean_squared_error(y_train, y_train_pred), mean_squared_error(y_test, y_test_pred),
r2_score(y_train, y_train_pred), r2_score(y_test, y_test_pred)))
```

```
Pipeline(steps=[('standardscaler', StandardScaler()), ('ridge', Ridge())])
MSE train: 22.547, test: 21.549
R^2 train: 0.743, test: 0.711
```

Example: LASSO

```python
# load and summarize the housing dataset
import pandas as pd
from matplotlib import pyplot
from numpy import mean
from numpy import std
from numpy import absolute
from sklearn.linear_model import Lasso
dataframe = pd.read_csv("housing.csv", header=None) # load dataset
data = dataframe.values
X, y = data[:, :-1], data[:, -1]
model = Lasso(alpha=1.0)                            # define model
model.fit(X, y)                                     # fit model
# define new data
row = [0.00632,18.00,2.310,0,0.5380,6.5750,65.20,4.0900,1,296.0,15.30,396.90,4.98]
yhat = model.predict([row])                         # make a prediction
print('Predicted: %.3f' % yhat)                     # summarize prediction
```

Reference:
https://machinelearningmastery.com/lasso-regression-with-python/

```
Lasso()
Predicted: 30.998
```

Example: ANN Regression

Reference:
https://thinkingneuron.com/using-artificial-neural-networks-for-regression-in-python/

```python
import pandas as pd
import numpy as np
# To remove the scientific notation from numpy arrays
np.set_printoptions(suppress=True)
CarPricesDataNumeric=pd.read_pickle('CarPricesData.pkl')
CarPricesDataNumeric.head()
# Separate Target Variable and Predictor Variables
TargetVariable=['Price']
Predictors=['Age', 'KM', 'Weight', 'HP', 'MetColor', 'CC', 'Doors']
X=CarPricesDataNumeric[Predictors].values
y=CarPricesDataNumeric[TargetVariable].values
from sklearn.preprocessing import StandardScaler
PredictorScaler=StandardScaler()
TargetVarScaler=StandardScaler()
```

	Age	KM	Weight	HP	MetColor	CC	Doors	Price
0	23.0	46986	1165.0	90	1	2000.0	3	13500
1	23.0	72937	1165.0	90	1	2000.0	3	13750
2	24.0	41711	1165.0	90	1	2000.0	3	13950
3	26.0	48000	1165.0	90	0	2000.0	3	14950
4	30.0	38500	1170.0	90	0	2000.0	3	13750

Example: ANN Regression

```
# Storing the fit object for later reference
PredictorScalerFit=PredictorScaler.fit(X)
TargetVarScalerFit=TargetVarScaler.fit(y)
X=PredictorScalerFit.transform(X)  # Standardize X
y=TargetVarScalerFit.transform(y)  # Standardize y
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X, y, test_size=0.3, random_state=42)
import torch
import torch.nn as nn
from sklearn import metrics
# input size, hidden unit, output size, batch size, and the learning rate.
n_input, n_hidden, n_out, batch_size, learning_rate = 7, 5, 1, 100, 0.01
# randomly initialize the dummy input and the output target data (or tensor)
data_x = torch.FloatTensor(X_train)
data_y = torch.FloatTensor(y_train).float()
# define ANN model
model = nn.Sequential(nn.Linear(n_input, n_hidden), nn.ReLU(),
                      nn.Linear(n_hidden, n_out), nn.ReLU())
loss_function = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

Example: ANN Regression

```
losses = []   # Training Loop
for epoch in range(5000):
    pred_y = model(data_x)
    loss = loss_function(pred_y, data_y)
    losses.append(loss.item())
    model.zero_grad()
    loss.backward()
    optimizer.step()
targets = model(torch.FloatTensor(X_test))
print(metrics.mean_squared_error(np.round(targets.detach().numpy()), y_test))
```

```
0.38377633018477214
```

Example: SVR

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
dataset = pd.read_csv('Position_Salaries.csv')
X = dataset.iloc[:,1:2].values.astype(float)
y = dataset.iloc[:,2:3].values.astype(float)
from sklearn.preprocessing import StandardScaler
sc_X = StandardScaler()
sc_y = StandardScaler()
X = sc_X.fit_transform(X)
y = sc_y.fit_transform(y)
from sklearn.svm import SVR
regressor = SVR(kernel='rbf')
regressor.fit(X,y)
y_pred = regressor.predict(np.array(6.5).reshape(-1,1))
print('Predicted value: ', y_pred)
```

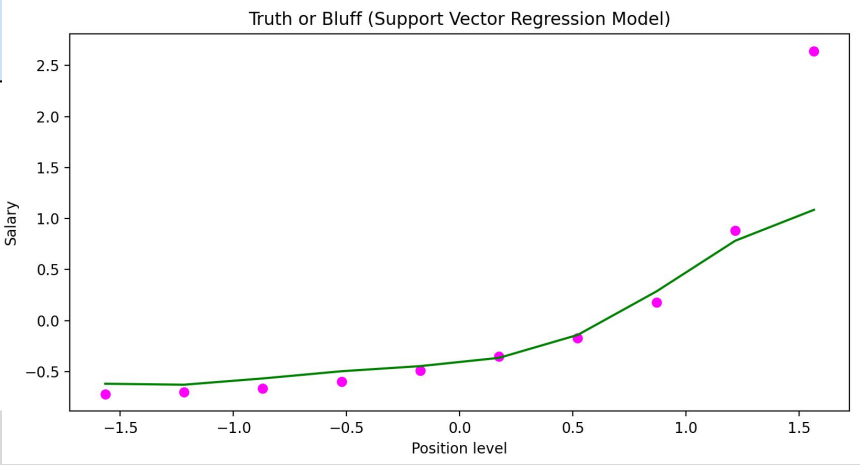
```
SVR()
Predicted value:  [0.01158103]
```

Reference:
https://medium.com/pursuitnotes/support-vector-regression-in-6-steps-with-python-c4569acd062d

Example: SVR

```
plt.scatter(X, y, color = 'magenta')
plt.plot(X, regressor.predict(X), color = 'green')
plt.title('Truth or Bluff (Support Vector Regression Model)')
plt.xlabel('Position level')
plt.ylabel('Salary')
plt.show()
```

```
<matplotlib.collections.PathCollection object at 0x000002EEEE25AA00>
[<matplotlib.lines.Line2D object at 0x000002EEEE25A910>]
Text(0.5, 1.0, 'Truth or Bluff (Support Vector Regression Model)')
Text(0.5, 0, 'Position level')
Text(0, 0.5, 'Salary')
```

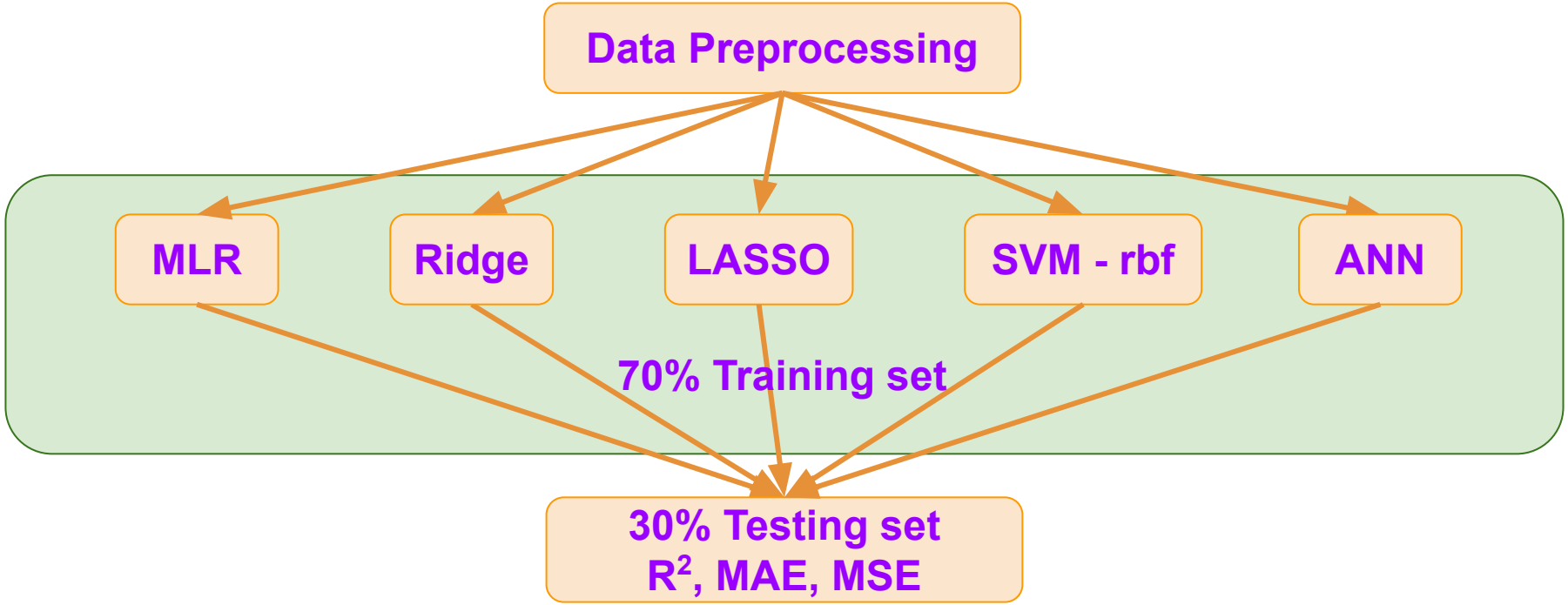


Parameters, Hyperparameters of Different Regressors

	Parameters	Hyperparameters
Multiple Linear Regression	Coefficient of X-variable and bias	No
Logistic Regression	Coefficient of X-variable and bias	Solver, penalty, and regularization strength
Ridge Regression	Coefficient of X-variable and bias	Lambda (tuning) controls the weighting of the penalty to the loss function
LASSO Regression	Coefficient of X-variable, bias and L1 regularization	Lambda (tuning) controls the weighting of the penalty to the loss function

Comparing Different Regression Models

- Goal: A fair comparison between different regression algorithms



Comparing Different Regression Models

- Dependent variable: sales
- Independent variables: age, gender, miles, debt, income

	age	gender	miles	debt	income	sales
count	963.000000	963.000000	963.000000	963.000000	963.000000	963.000000
mean	37.971963	0.512980	27.704050	14109.004154	6176.047767	11689.860852
std	12.290838	0.500091	13.378181	18273.702481	3260.670142	8986.896921
min	19.000000	0.000000	10.000000	0.000000	0.000000	500.000000
25%	27.000000	0.000000	20.000000	1475.000000	3506.500000	3554.000000
50%	37.000000	1.000000	25.000000	6236.000000	6360.000000	9130.000000
75%	49.000000	1.000000	32.000000	16686.000000	8649.500000	19245.000000
max	60.000000	1.000000	97.000000	59770.000000	11970.000000	29926.000000

Reference: https://github.com/MGCodesandStats/datasets

Comparing Different Regression Models - Steps

1. Load libraries and data
2. Drop rows containing NAN
3. Use `StandardScaler()` to normalize the data
4. Split into training (70%) and testing set (30%)
5. Run the data on 5 different models and evaluate the performance

Comparing Different Regression Models - Results

	R^2	MAE	MSE
Linear Regression	0.836060821919540	0.316506581093023	0.166663024523914
Ridge Regression	0.836062825867327	0.316538678987636	0.166660987280571
LASSO Regression	-0.00080718648223	0.884640127023149	1.017435579569234
SVR - rbf Kernel	0.894690176003710	0.239795094010764	0.107059544794648
ANN (5, 12 ReLu, 8 ReLu, 1)	0.889572802324507	0.252556553973129	0.112261943543874